A Project Report on

OWASP AI Model Watermarking

Submitted by

Sami Daire (Roll no. 231813)
Mohamed Ansari (Roll no. 231803)
Arfat Shaikh (Roll no. 231846)
Yash Bandiwadekar (Roll no. 231810)

in partial fulfillment for the award of the degree

BACHELOR OF ENGINEERING

in

Computer Science and Engineering (IOTCSBCT)

Under the Guidance of

Dr. Mohd. Shafi Pathan

Anjuman I Islam's
M. H. Saboo Siddik College of Engineering, Mumbai
University of Mumbai

2026 - 2027

CERTIFICATE

This is to certify that Sami Daire, Mohamed Ansari, Arfat Shaikh, Yash Bandiwadekar are the bonafide students of M. H. Saboo Siddik College of Engineering, Mumbai. They have successfully carried out the project titled "OWASP AI Model Watermarking" in partial fulfilment of the requirement of B. E. Degree in Computer Science and Engineering (IOTCSBCT) of Mumbai University during the academic year 2026-2027. The work has not been presented elsewhere for the award of any other degree or diploma prior to this.

……
(Dr. Mohd. Shafi Pathan)

……	……
(Er. Samana Jafri)	(Dr. Mohd. Shafi Pathan)
Co-ordinator CSE(IOTCSBCT)	Principal

Project Report Approval for B.E.

This project entitled "OWASP AI Model Watermarking" by Sami Daire, Mohamed Ansari, Arfat Shaikh, Yash Bandiwadekar is approved for the degree of Bachelor of Engineering in Computer Science and Engineering (IOTCSBCT) from University of Mumbai.

Examiners

1. - - - - - - - - - - - - - - - - - -

2. - - - - - - - - - - - - - - - - - -

Date:

Place:

ACKNOWLEDGEMENT

We are thankful to a number of individuals who have contributed towards our final year project and without their help it would not have been possible. Firstly, we offer our sincere thanks to our project guide, Dr. Mohd. Shafi Pathan, for constant and timely help and guidance throughout our preparation.

We are grateful to all project co-ordinators for their valuable inputs to our project. We are also grateful to the college authorities and the entire faculty for their support in providing us with the facilities required throughout this semester.

We are also highly grateful to Er. Samana Jafri, Co-ordinator CSE(IOTCSBCT), Principal, Dr. Mohd. Shafi Pathan, and Director Dr. Mohiuddin Ahmed for providing the facilities, a conducive environment and encouragement.

Signatures of all the students in the group

(Sami Daire)

(Mohamed Ansari)

(Arfat Shaikh)

(Yash Bandiwadekar)

Declaration

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in this submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Signatures of all the students in the group

(Sami Daire)

(Mohamed Ansari)

(Arfat Shaikh)

(Yash Bandiwadekar)

Abstract

Trained neural networks are valuable intellectual property, and model watermarking lets an owner show, after a theft, that a copied model is theirs. Recent work proves such ownership claims with zero-knowledge (ZK) proofs so that the secret watermark key is never revealed. We show that this privacy creates a weakness: when the watermark key and signature are not committed in advance, anyone holding a copy of the model can fit a counterfeit key that extracts a signature of their own choice, and the resulting ZK proof verifies under the genuine owner's verification key. We demonstrate this forgery against an uncommitted EZKL proof of white-box (Uchida-style) watermark extraction.

We then present Sigil, a protocol and open implementation in which both the watermark key and the signature are derived from a secret seed committed at embedding time, the commitment is bound to an Ed25519 owner identity and a registry timestamp, the bit-error threshold is enforced inside the circuit, and every proof is bound to a hash of the suspect model and a verifier-issued nonce. The statement is implemented as a Circom circuit of 116,822 constraints proved with Groth16; on a single workstation a proof takes about 2.2 s. On an MNIST model the watermark costs 0.03 percentage points of accuracy, survives pruning, quantisation and fine-tuning, and a 100,000-seed forgery search found no accepted seed. Feature permutation remains an open weakness. The system is delivered as a Python SDK, a command-line tool and a local web dashboard, aligned with the OWASP AI Model Watermarking project.

Keywords: Model Watermarking, Zero-Knowledge Proofs, zk-SNARK, Groth16, Circom, Ownership Verification, Ambiguity Attack, OWASP, AI Security

Contents

Chapter 1 Introduction 1
1.1 Motivation 1
1.2 Problem Statement 2
1.3 Objectives and Scope 2
1.4 Methodology 3
1.5 Organization of Project Report 4
Chapter 2 Literature Review 5
2.1 OWASP Context 6
2.2 Embedding Watermarks in Neural Networks 7
2.2.1 White-box watermarks 7
2.2.2 Black-box watermarks 7
2.3 Removal Attacks and Robustness 8
2.4 Ownership Forgery and Ambiguity Attacks 8
2.5 Committed and Hash-Derived Watermark Keys 8
2.6 Zero-Knowledge Proof Systems 9
2.7 Zero-Knowledge Verified Ownership 9
2.8 Fingerprinting of Large Language Models 10
2.9 Summary of Literature 10
2.10 Research Gaps 11
Chapter 3 Theoretical Background and Design Methodology 13
3.1 Threat Model and Requirements 13
3.2 System Architecture 14
3.3 Watermark Embedding 15
3.4 Committed-Seed Ownership Protocol 17
3.4.1 Why a commitment is not enough 17
3.4.2 Primitives and derivations 17
3.4.3 Protocol flow 17
3.5 Zero-Knowledge Circuit 18
3.5.1 Trusted setup 20
3.6 Verification Procedure 20
3.7 Software Implementation 21
3.7.1 Python SDK and command-line tool 21
3.7.2 Web dashboard 22
3.8 Software Security Design 23

Chapter 4 Experimental Setup and Results 24
4.1 Experimental Environment 24
4.2 Fidelity of the Watermark (Demo A) 25
4.3 Robustness to Removal Attacks (Demo B, RQ2) 25
4.4 Forgery Without a Commitment (Demo C, RQ1) 26
4.5 Committed-Seed Defence (Demo D, RQ1) 26
4.6 ZK Proofs with EZKL (Demo E) 27
4.7 End-to-End Protocol (Demo F) 28
4.8 Cost of the Circom Circuit (RQ3) 29
4.9 Web Dashboard 29
4.10 Discussion and Limitations 33
Chapter 5 Conclusion 35
5.1 Conclusion 35
5.2 Future Scope 36
References 37
Appendix-I: Protocol Configuration and Notation 40
Appendix-II: Copies of the Published Papers 41
Appendix-III: Experimental Data 42
Appendix-IV: Installation and User Guide 43

List of Figures

Figure 1.1: Project methodology....4
Figure 2.1: Taxonomy of model ownership verification research....6
Figure 3.1: Threat model: owner, thief, registry and verifier....14
Figure 3.2: System architecture of Sigil....15
Figure 3.3: Watermark embedding and extraction pipeline....16
Figure 3.4: Sequence diagram of registration, claim, proof and verification....18
Figure 3.5: Block diagram of the ownership circuit....20
Figure 3.6: Data flow diagram (Level 1)....22
Figure 3.7: Use case diagram....23
Figure 4.1: Accuracy and owner BER under removal attacks....26
Figure 4.2: Distribution of BER over the thief's seed search (D2)....27
Figure 4.3: Dashboard: landing page....30
Figure 4.4: Dashboard: overview with headline results....30
Figure 4.5: Dashboard: embedding the watermark....31
Figure 4.6: Dashboard: ownership verification verdict....31
Figure 4.7: Dashboard: robustness lab....32
Figure 4.8: Dashboard: Demo C, ownership forgery without a commitment....32
Figure 4.9: Dashboard: Demo D, committed-seed defence....33
Figure 4.10: Dashboard: Demo F, protocol v2 ownership proof....33

List of Tables

Table 2.1: Summary of literature survey 10
Table 2.2: Comparison of ZK ownership systems with this project 11
Table 2.3: Research gaps identified from the literature 11
Table 3.1: Parties in the protocol and their trust assumptions 13
Table 3.2: Protocol derivations (H = Poseidon) 17
Table 3.3: Checks performed by the verifier 21
Table 3.4: Technology stack 21
Table 3.5: STRIDE threat model of the software (summary) 23
Table 4.1: Experimental environment 24
Table 4.2: Accuracy and owner bit error rate 25
Table 4.3: Robustness of the watermark under removal attacks ($\theta = 0.125$) 25
Table 4.4: Forgery success under the three commitment variants 26
Table 4.5: Cost of EZKL proofs (owner and thief runs) 27
Table 4.6: Rejection of misuse attempts in protocol v2 28
Table 4.7: Robustness of the ZK ownership proof (protocol v2) 28
Table 4.8: Cost of the protocol v2 ownership circuit 29

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
BER	Bit Error Rate
BN254	Barreto–Naehrig pairing-friendly curve (254-bit)
CLI	Command-Line Interface
CNN	Convolutional Neural Network
CSRF	Cross-Site Request Forgery
DFD	Data Flow Diagram
FTAL	Fine-Tune All Layers
GPU	Graphics Processing Unit
IP	Intellectual Property
KZG	Kate–Zaverucha–Goldberg polynomial commitment
MNIST	Modified National Institute of Standards and Technology (digit dataset)
ML	Machine Learning
ONNX	Open Neural Network Exchange
OWASP	Open Worldwide Application Security Project
PK	Proving Key
PRG	Pseudo-Random Generator
R1CS	Rank-1 Constraint System
RTAL	Re-Train All Layers
SDK	Software Development Kit
SNARK	Succinct Non-interactive Argument of Knowledge
SoK	Systematization of Knowledge
SRS	Structured Reference String
SSE	Server-Sent Events
STRIDE	Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
UI	User Interface
VK	Verification Key
ZK	Zero Knowledge
ZKP	Zero-Knowledge Proof

Chapter 1

Introduction

1.1 Motivation

Training a modern neural network costs data, compute and expertise, so a trained model is a valuable asset. Once a model's weights leave the owner's control, through an insider, a compromised server or a leaked file, the thief can deploy or resell an exact copy. The OWASP Machine Learning Security Top 10 lists model theft as risk ML05:2023 and names watermarking among its preventive measures [1], and the OWASP AI Exchange describes watermarking as a way "to demonstrate ownership after a model has been stolen or replicated, rather than to prevent the theft itself" [2].

A white-box watermark hides a short binary signature in a model's weights; only someone with the secret key can read it back [3]. The difficulty is the dispute itself. If the owner reveals the key to a judge or verifier, the key becomes public and the watermark can be removed. Zero-knowledge (ZK) proofs offer a way out: the owner proves that the extraction succeeds without revealing the key [4]. OWASP runs an incubator project, *OWASP AI Model Watermarking*, whose stated goal is exactly this kind of ZK-proof-based ownership verification, but its repository currently contains only website content and no implementation [5], [6].

This project started from that gap and found a deeper problem. A ZK proof hides the key from the verifier, which also hides whether the key was *chosen after seeing the model*. The OWASP AI Exchange asks that watermarks "avoid ambiguity where multiple parties could plausibly claim ownership of the same model" [2]. Our motivation is to build an open, working ownership-verification system that satisfies this requirement even when the person making the claim is dishonest.

Note on affiliation. This is a student final-year project aligned with the OWASP AI Model Watermarking incubator project. It is not official OWASP work and is not endorsed by OWASP.

1.2 Problem Statement

To design and implement a white-box neural-network watermarking system whose ownership claims can be verified with a zero-knowledge proof that reveals nothing about the watermark, and that remains sound against a malicious claimant who holds a copy of the model and can fit counterfeit keys, replay proofs or overwrite the watermark.

1.3 Objectives and Scope

The objectives, refined from the project's Statement of Purpose, are:

1. Embed a cryptographic watermark into the weights of a neural network without measurable loss of accuracy, for PyTorch, Keras and ONNX models.
2. Demonstrate, with measurements, that uncommitted ZK ownership proofs can be forged.
3. Design a committed-seed protocol in which the watermark key and signature are both derived from a seed that is committed, signed and timestamped at embedding time.
4. Implement the ownership statement as a zk-SNARK circuit (Circom and SnarkJS, Groth16) that enforces the bit-error threshold in-circuit and binds each proof to the suspect model and to a fresh challenge.
5. Evaluate robustness of the watermark, and of the ZK proof, against pruning, quantisation, fine-tuning, overwriting and permutation attacks.
6. Deliver a framework-agnostic Python SDK, a command-line tool and a local web dashboard for embedding, proving and verifying.

In scope: white-box watermarks embedded in model weights, used for ownership verification after theft. **Out of scope:** watermarks on model outputs (generated text, code or images), and model extraction through query-only (API) access, which the OWASP text itself identifies as a limitation of typical watermarks [2].

1.4 Methodology

The work followed the phased method shown in Figure 1.1:

1. **Literature survey and gap analysis.** Every cited paper was opened and read; the depth of reading is recorded. The OWASP AI Exchange text was turned into two project requirements: R1 (robustness) and R2 (non-ambiguity).
2. **Baseline and attack study.** An Uchida-style watermark was trained into an MNIST convolutional network and attacked with a subset of the removal attacks from the SoK of Lukas et al. [7].
3. **Forgery study.** A counterfeit-key attack was implemented and run against an uncommitted EZKL proof to test requirement R2.
4. **Protocol design.** A committed-seed protocol was specified (Chapter 3) and its security argued against a malicious prover.
5. **Circuit and SDK implementation.** The ownership statement was written in Circom, keys were generated with a public Powers of Tau ceremony, and the protocol was packaged as the *sigil* SDK.
6. **Evaluation.** Fidelity, robustness, forgery resistance, misuse rejection and cost were measured on a single workstation and recorded as machine-readable results.
7. **Product.** A local web dashboard replays or runs every experiment live.

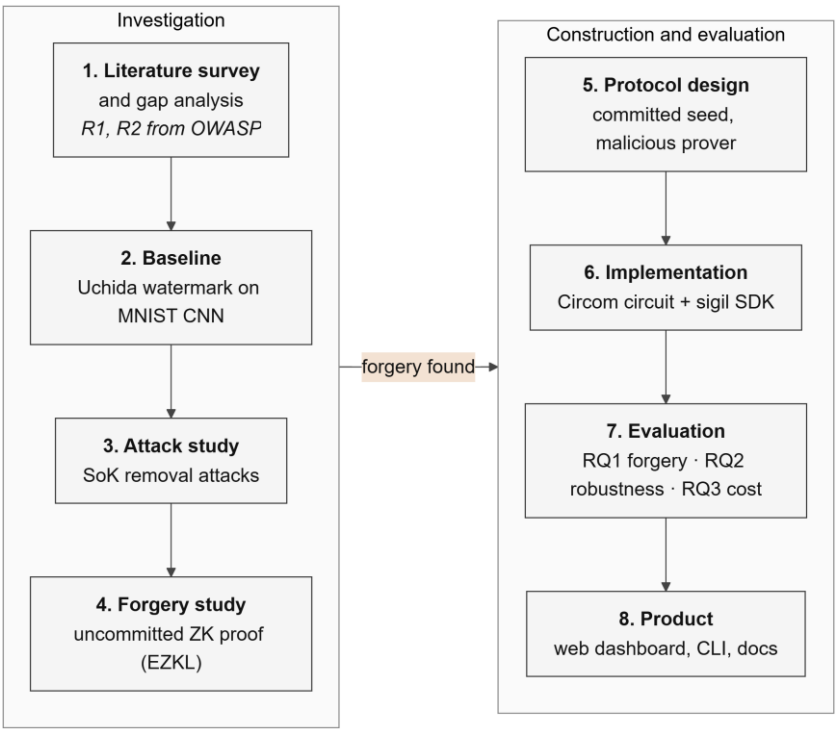


Figure 1.1: Project methodology

1.5 Organization of Project Report

This project report is organized as follows:

- Chapter 2 presents the literature survey on model watermarking, removal and forgery attacks, committed and hash-derived keys, zero-knowledge proof systems and ZK-verified ownership, and ends with the research gaps.
- Chapter 3 explains the theoretical background and design methodology: threat model, system architecture, watermark embedding, the committed-seed protocol, the ZK circuit and the software implementation.
- Chapter 4 is dedicated to the experimental setup and results: fidelity, robustness, forgery, the protocol demonstrations and cost.
- Chapter 5 presents the conclusions and future scope for this project.

Chapter 2

Literature Review

This chapter surveys the work this project builds on. The survey followed one rule: a claim is attributed to a paper only if that paper was opened and read, and statements taken only from an abstract are not extended with details from memory. Papers were found by keyword search, by reading the references of core papers, and by a forward-citation sweep of ZKROWNN [4] using the Semantic Scholar API, which returned about 13 citing works. Figure 2.1 places the surveyed work in a taxonomy.

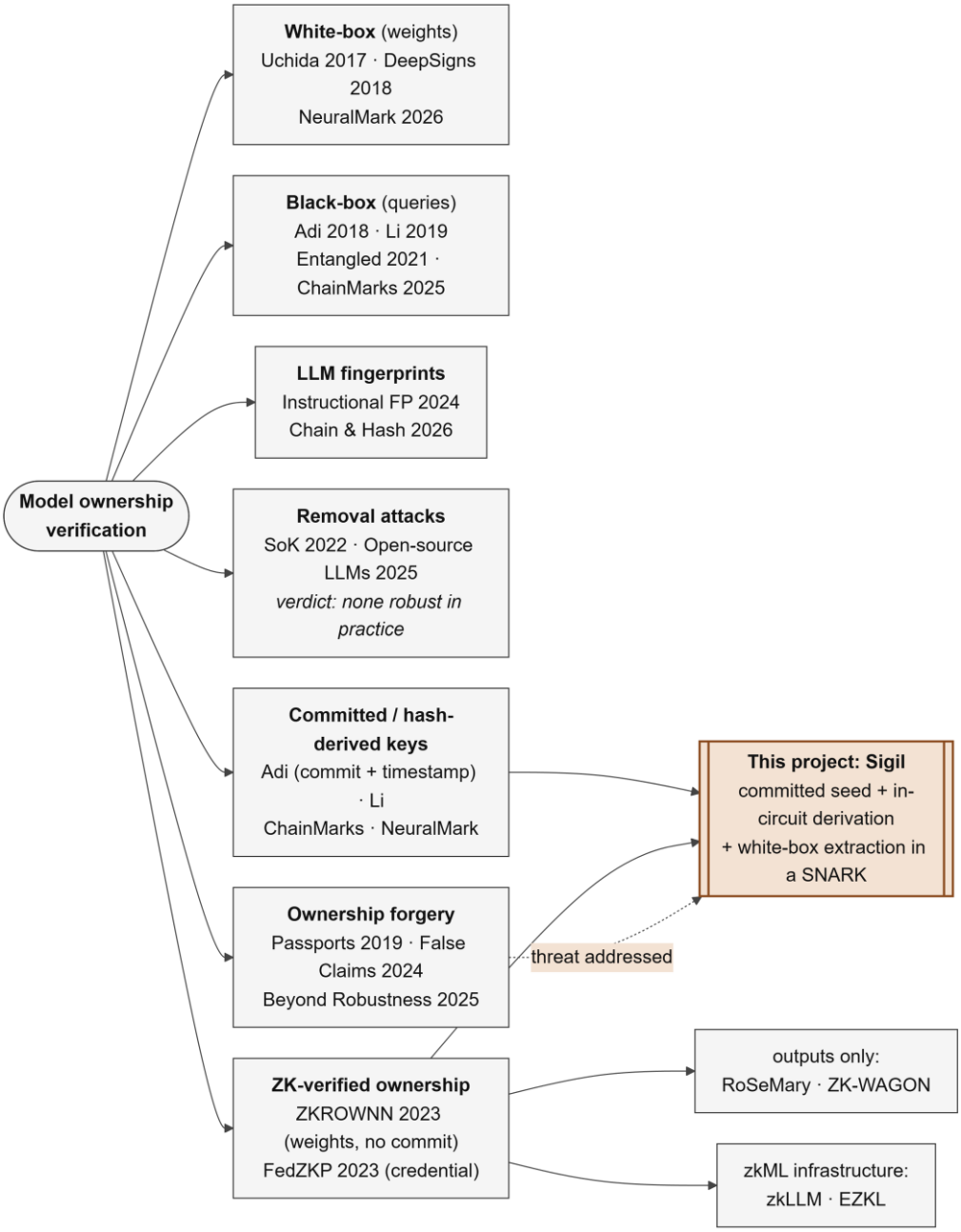


Figure 2.1: Taxonomy of model ownership verification research

2.1 OWASP Context

OWASP AI Model Watermarking is an incubator project whose objectives include ZK-proof-based watermarking for several model types, ZK ownership verification protocols, an open-source extensible application, and resilience research against attacks; its success criteria include minimal performance impact and watermark survival against model transformation attacks [5]. Its repository holds only website files [6]. A related OWASP project, SecureML, addresses model *signing* through OpenSSF Model Signing and

Sigstore rather than ZK proofs [8]. The OWASP LLM Top 10 (2025) also recommends watermarking frameworks under LLM10, Unbounded Consumption, though for model outputs [9].

The OWASP AI Exchange control *#MODEL WATERMARKING* gives the wording this project is evaluated against [2]. It states that watermarking techniques should "remain detectable even if the model is modified (for example through fine-tuning or pruning)" and should "avoid ambiguity where multiple parties could plausibly claim ownership of the same model". We label these **R1 (robustness)** and **R2 (non-ambiguity)**; the labels are ours, not OWASP's. The same control notes that watermarks are limited against model exfiltration through queries, citing Entangled Watermarks [10].

2.2 Embedding Watermarks in Neural Networks

2.2.1 White-box watermarks

Uchida et al. introduced white-box watermarking: a regularisation term added during training embeds a bit string into the weights of one layer, and the bits are read back by projecting the weights with a secret key matrix. They report that the watermark survives fine-tuning and pruning of up to 65% of parameters [3]. DeepSigns embeds the watermark in the distribution of hidden-layer activations rather than in the weights [11]; it is the scheme ZKROWNN later proves in zero knowledge. NeuralMark (AAAI 2026) derives the watermark as a SHAKE-256 hash of a secret key and also uses the hash to select which parameters to embed in. It targets forging and overwriting attacks and is evaluated on 13 convolutional and Transformer architectures, but verification hands the key to a fully trusted third party [12].

2.2.2 Black-box watermarks

Black-box schemes are checked by querying the model. Adi et al. use a backdoor: a trigger set of inputs with chosen labels [13]. Their paper goes further than a trigger set. The verification key is a *commitment* to the trigger inputs and labels, which must be published before verification, for example on "a time-stamped bulletin board", and an appendix sketches a zero-knowledge argument for public verifiability; a practically efficient ZK proof is left open. Li et al. derive the watermark from SHA-256 hashes of an RSA signature over the owner's identifier and a global timestamp, embedded by "null embedding" at initial training [14]. Entangled Watermarks tie watermark features to task

features so that removal costs accuracy, and verify ownership with fewer than 100 queries [10]. ChainMarks generates trigger inputs by repeatedly hashing a secret key and derives target labels from the owner's digital signature [15].

2.3 Removal Attacks and Robustness

The most thorough robustness study is the SoK of Lukas et al. (IEEE S&P 2022), which evaluates 11 watermarking schemes, including Uchida, DeepSigns and Adi, against fine-tuning, pruning, fine-pruning, quantisation, weight shifting, neural cleanse, overwriting, distillation and retraining attacks. Its verdict is that "none of the surveyed watermarking schemes is robust in practice" [7]. The same concern is raised for open-weight language models, where watermarks must survive merging, quantisation and fine-tuning and current methods do not [16]. This literature motivates requirement R1 and shapes our attack suite in Chapter 4.

2.4 Ownership Forgery and Ambiguity Attacks

Robustness is only half of the problem; a watermark must also not be *forgeable*. Fan et al. describe ambiguity attacks that "aim to cast doubts on the ownership verification by forging counterfeit watermarks" and show they threaten existing DNN watermarking methods [17]. Liu et al. show that a malicious accuser can make false ownership claims against independent models using transferable adversarial examples, and demonstrate this against commercial services [18]. Bui and Tran report an "ownership deadlock" for a statistical watermark: strong regularisation gives the owner's key a p-value near 10^{-102}, yet a random key still reaches $p = 0.022$, and they call for cryptographically grounded designs [19]. Together these works establish forgery as a real threat to requirement R2.

2.5 Committed and Hash-Derived Watermark Keys

Several works defend against forgery by binding the watermark to the owner cryptographically: Adi et al. commit to and timestamp the trigger set [13]; Li et al. derive the watermark from a signature over identity and time [14]; ChainMarks uses a hash chain and a signature [15]; NeuralMark uses a hash of a secret key [12]; and for language models, Chain & Hash cryptographically binds fingerprint prompts to their responses [20]. None of these combines the binding with a zero-knowledge proof of white-box extraction: the key or signature is revealed to a verifier, or the scheme is black-box. This

project therefore does not claim commitment, timestamping or hash derivation as new ideas; it adopts them.

2.6 Zero-Knowledge Proof Systems

A zero-knowledge proof lets a prover convince a verifier that a statement is true while revealing nothing else; the notion was introduced by Goldwasser, Micali and Rackoff [21]. A zk-SNARK is a succinct, non-interactive argument of knowledge: the proof is small and fast to verify regardless of the size of the computation. Pinocchio made general-purpose verifiable computation nearly practical by compiling programs to arithmetic circuits [22]. Groth's 2016 construction gives pairing-based proofs of three group elements with very fast verification, at the cost of a circuit-specific trusted setup [23]; it is the proof system used in this project. Poseidon is a hash function designed to be cheap inside such circuits [24], and we use it for every commitment and key derivation. Circom is a domain-specific language that compiles circuits to rank-1 constraint systems [25], and SnarkJS provides setup, proving and verification for those circuits [26]. EZKL compiles ONNX models into Halo2 circuits with KZG commitments [27]; it is used in our early forgery experiments, although it has no open-source licence file, which matters for any OWASP contribution.

2.7 Zero-Knowledge Verified Ownership

ZKROWNN (DAC 2023) is the closest prior system [4]. It proves in Groth16 that a private key, applied to the public model's activations, extracts a DeepSigns signature with a bit error rate below a threshold. It reports a 127-byte proof and 1 ms verification for a CIFAR-10 CNN (590,624 constraints, 11.2 s proving) on a 128 GB workstation. Its threat model assumes a semi-honest prover that "will not deviate from the protocol"; the key, projection matrix and signature are private witnesses with no prior commitment, and removal attacks are not evaluated.

RoSeMary applies ZK-verifiable watermarking to code generated by language models, using a customised EZKL/Halo2 circuit; it reports 7.15 s proving with about 2.6 GB of RAM and evaluates content-level attacks such as variable renaming, but it watermarks outputs rather than weights and assumes an honest setup [28]. **ZK-WAGON** watermarks images from generative models and hides the proof in the image by LSB steganography; it reports no timing or robustness tables [29]. **FedZKP** embeds the hash of an xLPN

public instance into batch-normalisation weights in federated learning and gives an interactive ZK proof of knowledge of the secret, while the extraction and Hamming-distance check run in the clear [30]. **zkLLM** proves *inference* of large language models rather than watermark extraction, needing an A100 GPU and up to 23 GB of memory [31].

2.8 Fingerprinting of Large Language Models

For language models, fingerprinting is an alternative to embedded watermarks. Instructional fingerprinting implants an instruction backdoor and was tested on 11 LLMs [32]; Chain & Hash binds fingerprints with hashes and works on LoRA adapters [20]. A 2026 survey organises fingerprinting and watermarking across datasets, models and generated content [33]. We treat LLMs as future scope (Chapter 5).

2.9 Summary of Literature

Table 2.1 summarises the surveyed works, and Table 2.2 compares the ZK ownership systems with this project.

Table 2.1: Summary of literature survey

Sr.	Author(s), Year, Venue	Technique	Key finding	Limitation w.r.t. this project
1	Uchida et al., 2017, ICMR [3]	White-box weight regulariser	Survives fine-tuning, 65% pruning	Key revealed to verify; no ZK
2	Adi et al., 2018, USENIX Sec. [13]	Backdoor trigger set, committed and timestamped	ZK argument sketched	Black-box; no implemented ZK proof
3	Li et al., 2019, arXiv [14]	Watermark from RSA signature hashes	Resists piracy (second mark)	Signature revealed; no ZK
4	Fan et al., 2019, NeurIPS [17]	Passport layers	Ambiguity attacks forge counterfeit marks	Not ZK-based
5	Jia et al., 2021, USENIX Sec. [10]	Entangled watermarks	<100 queries to verify	Black-box; no ZK
6	Lukas et al., 2022, IEEE S&P [7]	SoK of 11 schemes vs attacks	No scheme robust in practice	Evaluation only
7	Sheybani et al., 2023, DAC [4]	Groth16 proof of DeepSigns extraction	127 B proof, 1 ms verify	Semi-honest prover; no commitment; no attack evaluation
8	Yang et al., 2023, arXiv [30]	Hash-derived mark + xLPN ZK proof	Federated ownership in ZK	Extraction checked in the clear
9	Liu et al., 2024, USENIX Sec. [18]	False claims via adversarial examples	Malicious accusers succeed	Attack paper
10	Sun et al., 2024, CCS [31]	ZK proofs of LLM inference	Feasible up to 13B params	Inference, not watermarks; A100 needed

Sr.	Author(s), Year, Venue	Technique	Key finding	Limitation w.r.t. this project
11	Choi et al., 2025, ASIA CCS [15]	Hash-chain triggers, signed labels	Resists removal and ambiguity	Black-box; no ZK
12	Zhang et al., 2025/26, TAISAP [28]	ZK-verifiable code watermark (EZKL)	7.15 s prove, 2.6 GB	Outputs only; honest setup assumed
13	Ramakrishnan et al., 2025, AIMLSystems [29]	ZK proof for image generators	Proof hidden in image	No robustness or cost results
14	Bui & Tran, 2025, VCRIS [19]	Unforgeability analysis	Random key reaches p = 0.022	Analysis only
15	Yao et al., 2026, AAAI [12]	Hashed watermark as filter	Defeats forging, overwriting	Trusted verifier sees key; no ZK

Table 2.2: Comparison of ZK ownership systems with this project

Property	ZKROWNN [4]	RoSeMary [28]	FedZKP [30]	Sigil (this project)
What is watermarked	Model activations	Generated code	Model weights (FL)	Model weights
Statement proved in ZK	Extraction, BER ≤ θ	Decoder extraction, BER ≤ θ	Knowledge of credential	Seed opening + derivation + extraction, BER ≤ θ
Key committed in advance	No	Decoder in VK	Hash in weights	Yes (Poseidon, signed, timestamped)
Signature derived from secret	No	No	Yes	Yes (in-circuit)
Proof bound to suspect model	Model is public input	—	—	Yes (weight hash)
Replay protection	Not stated	Not stated	Interactive	Verifier nonce
Prover model	Semi-honest	Malicious user, honest setup	—	Malicious
Removal attacks evaluated	No	Content-level	Fine-tune, prune	Prune, quantise, fine-tune, overwrite, permute
Proof system	Groth16	Halo2/KZG	Σ-protocol	Groth16 (Circom)

2.10 Research Gaps

The survey leads to the gaps in Table 2.3. Gap G1 is the core of this project.

Table 2.3: Research gaps identified from the literature

ID	Gap	Evidence	Addressed in this project by
G1	Uncommitted ZK ownership proofs are forgeable, and ZK hides the forgery; ZKROWNN assumes a semi-honest prover	[4], [17], [18], [19]	Committed-seed protocol with in-circuit derivation of key and signature (Ch. 3); forgery measured (Ch. 4)
G2	ZK-proved extraction from weights has not been tested against removal attacks	[7], [4]	Attack suite with proof accepted yes/no (Ch. 4)

ID	Gap	Evidence	Addressed in this project by
G3	Proofs are not bound to a specific suspect model	[4]	Public hash of the suspect weights checked by the verifier
G4	White-box marks do not cover query-only exfiltration	[2], [10]	Stated as a scope limit; future work
G5	No open, licence-clean implementation for the OWASP project	[6], [27]	Circom/SnarkJS implementation and SDK
G6	ZK verification for LLM-scale models	[16], [31]	Future scope

Chapter 3

Theoretical Background and Design Methodology

This chapter describes the threat model, the architecture of the Sigil system, the watermark embedding method, the committed-seed protocol and its zero-knowledge circuit, and the software that implements them.

3.1 Threat Model and Requirements

Table 3.1 lists the parties. The crucial difference from ZKROWNN [4] is that the **prover is malicious**: anyone may claim ownership of any model, and the system must still reject false claims.

Table 3.1: Parties in the protocol and their trust assumptions

Party	Holds	Trusted for
Owner	Secret seed s, Ed25519 private key	Nothing (may be malicious when claiming another's model)
Registry	Ed25519 registry key	Issuing honest timestamps (receipts)
Verifier	Registry public keys, the suspect model file	Nothing beyond running the verification checks
Thief	Stolen weights; may register their own seed and key; may fine-tune, prune, overwrite	—

White-box access to the suspect model is required. From the OWASP text and the gaps we derive the requirements used throughout the evaluation:

- **R1 Robustness:** the watermark remains detectable after fine-tuning, pruning and similar modifications [2].

- **R2 Non-ambiguity:** two parties cannot both convincingly claim the same model; in particular, a claimant cannot fit a key after seeing the model.
- **Zero knowledge:** a proof reveals nothing about the key or signature.
- **Binding:** a proof is valid only for one registration, one suspect model and one verification session.

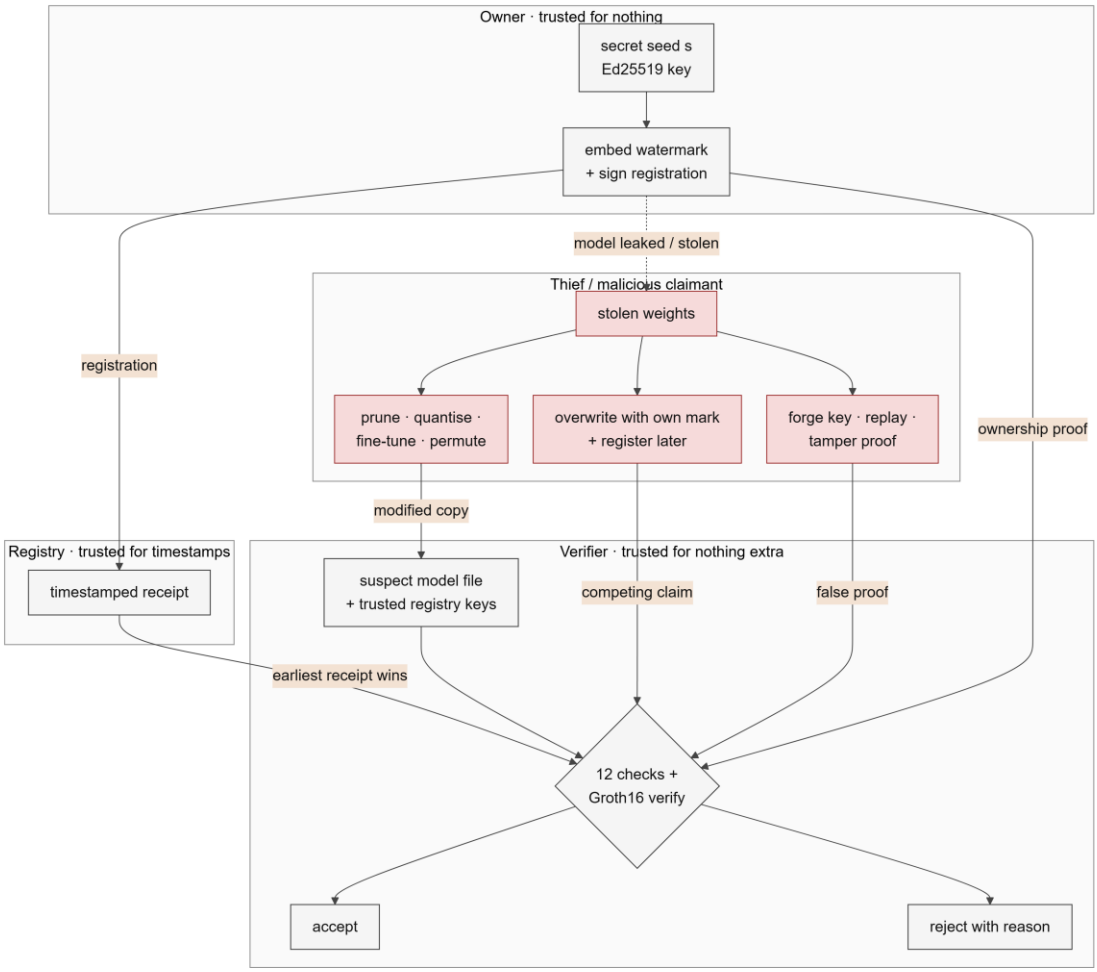


Figure 3.1: Threat model: owner, thief, registry and verifier

3.2 System Architecture

Figure 3.2 shows the architecture. Sigil is a local application: a Python server, using only the standard library and bound to 127.0.0.1, serves a prebuilt React user interface and runs the experiment scripts one job at a time, streaming their output to the browser with server-sent events. The *sigil* SDK implements the protocol, the Poseidon hash, Ed25519 identities and model adapters, and calls Circom and SnarkJS for proving. Results are written as JSON files that the interface renders, so a recorded run can be replayed on a machine without the ML stack.

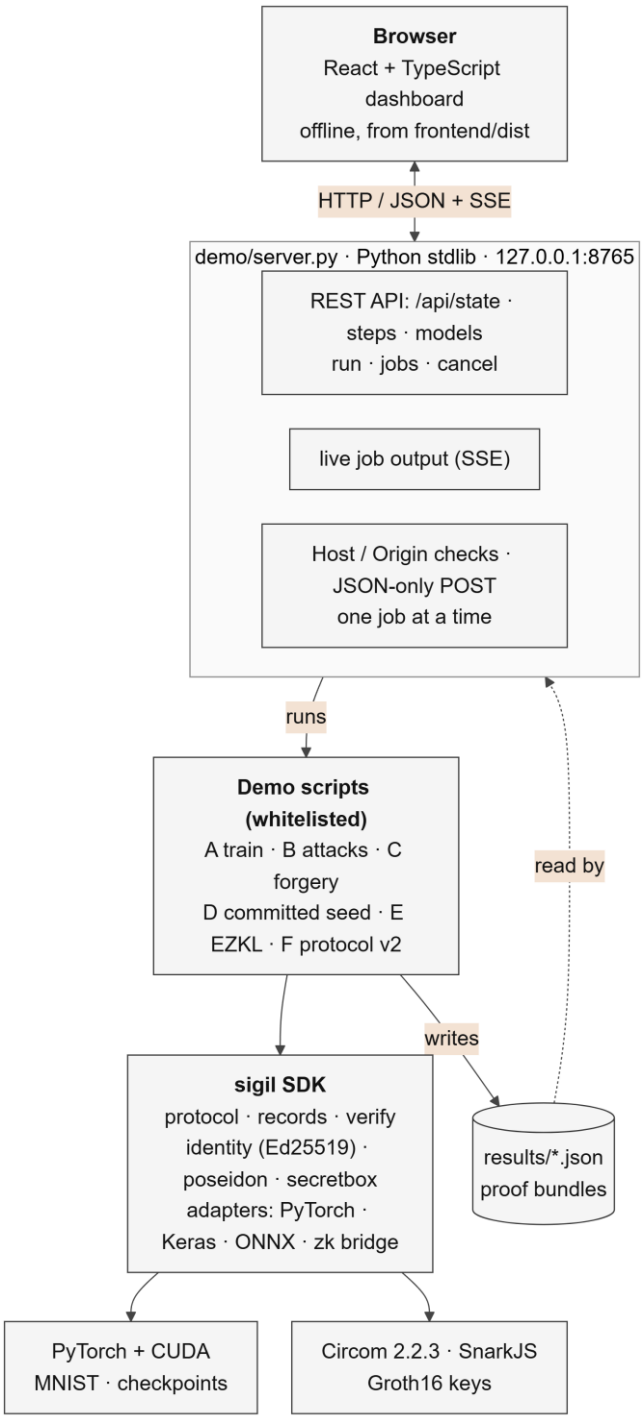


Figure 3.2: System architecture of Sigil

3.3 Watermark Embedding

We use an Uchida-style white-box watermark [3]. Let $w \in \mathrm{R}^{\mathrm{M}}$ be the *Uchida vector* of a chosen layer: the mean of the layer's weights over its output units, flattened in a canonical order. For the convolution layer *conv2* of our MNIST network, M = 288. A key matrix X

$\in \{-1, +1\}^{T \times M}$ and a signature $b \in \{0, 1\}^T$ of T bits are derived from the owner's seed (Section 3.4). Embedding adds a regulariser to the training loss:

$$L = L_{task} + \lambda \cdot \Sigma_i \, BCE(\sigma(X_i \cdot w), b_i)$$

where σ is the sigmoid and BCE is binary cross-entropy, so that the projections of w onto the key rows take the signs encoded by b. Extraction quantises the weights to fixed point, $w_q = \text{round}(w \cdot 2^{16})$, and reads

$$bit_i = 1 \;\text{ if }\; \Sigma_j \, X_{ij} \cdot w_{q,j} \geq 0, \;\text{ otherwise } 0$$

The claim is accepted if at most max_err = 8 of the T = 64 bits disagree with b, i.e. a bit error rate threshold $\theta = 8/64 = 0.125$. A parity study over five seeds found that the ±1 key used by the circuit behaves like Uchida's original Gaussian key (Appendix III). The SDK computes the same vector from PyTorch (.pt), Keras (.h5/.keras) and ONNX files by transposing each framework's weight layout to a canonical order.

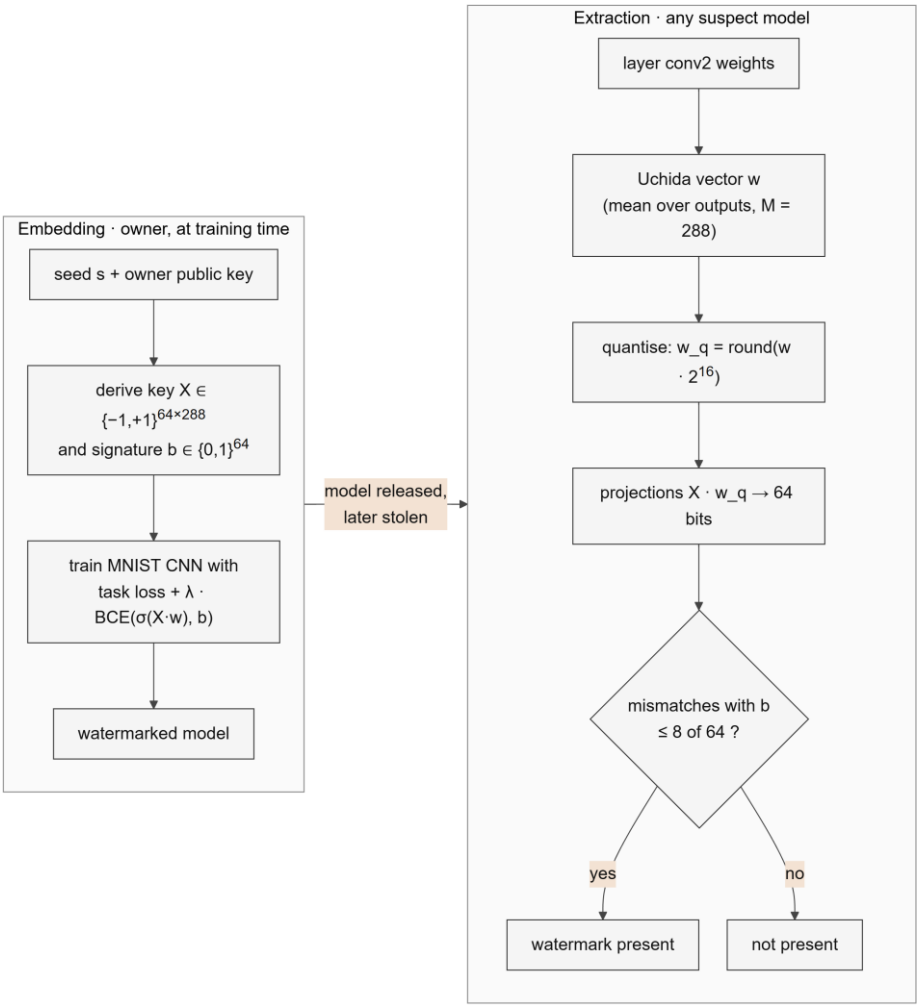


Figure 3.3: Watermark embedding and extraction pipeline

3.4 Committed-Seed Ownership Protocol

3.4.1 Why a commitment is not enough

If the key X is secret and uncommitted, a thief holding the weights can choose any random X′ and simply *define* the signature b′ = extract(X′, w), obtaining zero bit errors on any model. In ZK the verifier never sees X′, so nothing distinguishes it from a genuine key. Committing to the key alone does not help: if the signature can still be chosen freely, the thief commits to a random seed, derives X′, and again sets b′ to whatever the extraction returns. Our experiments confirm both attacks (Chapter 4). The protocol therefore derives **both** the key and the signature from the committed seed, so a thief's only option is to search over seeds, each of which succeeds with only the null false-positive probability.

3.4.2 Primitives and derivations

All hashes are Poseidon over the BN254 scalar field [24], with domain tags separating their uses. Owner identities are Ed25519 key pairs [34]. Table 3.2 gives the derivations.

Table 3.2: Protocol derivations (H = Poseidon)

Value	Definition
pk_hash	H(PK, first half of owner public key, second half)
config_hash	H(CFG, 2, T, M, scale_bits, w_bits, max_err, tag(layer), tag(norm), PRG_BITS)
seed s	248 uniformly random bits
Commitment C	H(COMMIT, s, pk_hash, config_hash)
Key X	X[i][j] = 2·keybit[i·M + j] − 1, keybits from low 200 bits of H(KEY, s, pk_hash, k), k = 0, 1, …
Signature b	First T bits of the stream H(SIG, s, pk_hash, k)
w_hash	Sponge of Poseidon over the quantised suspect vector (15 values per call), finalised with M

Because X and b depend on the owner's public key as well as the seed, a thief cannot grind identities against a fixed key, and because the configuration is inside the commitment, the threshold cannot be relaxed after the fact.

3.4.3 Protocol flow

The protocol has three phases, shown in Figure 3.4:

1. **Register (embedding time).** The owner generates an Ed25519 identity and a seed, derives X and b, embeds the watermark, and creates a signed *registration* record

containing the commitment C, the configuration and the model's SHA-256. A registry countersigns it with a timestamped *receipt*.

2. **Claim.** A verifier holding a suspect model opens a *claim* that names the registration and the suspect file and contains a fresh random nonce.
3. **Prove and verify.** The owner produces a Groth16 proof with public signals (pk_hash, config_hash, C, w_hash, nonce). The verifier recomputes every public signal itself, including w_hash from the suspect file, checks the signatures and timestamps, and verifies the proof.

Dispute rule. If a thief later embeds and registers their own watermark (overwriting), both parties may produce valid proofs; the registration with the earliest registry timestamp wins, and receipts cannot be back-dated without the registry key.

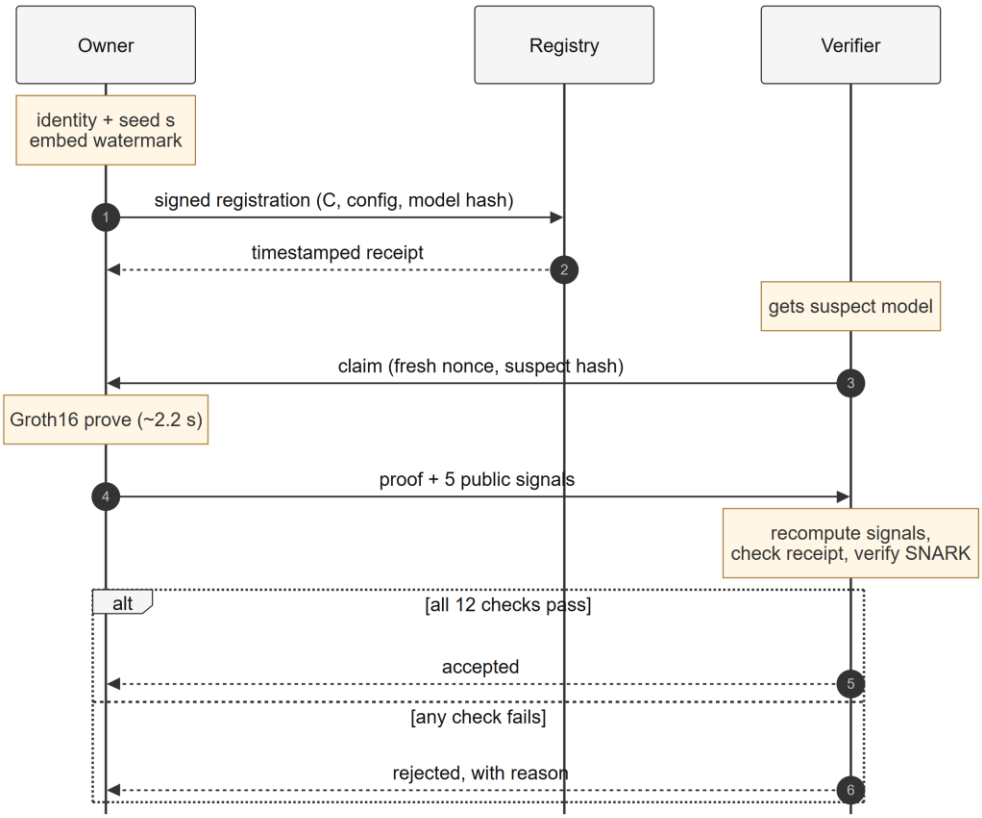


Figure 3.4: Sequence diagram of registration, claim, proof and verification

3.5 Zero-Knowledge Circuit

The ownership statement is written in Circom [25] and compiled for fixed parameters (T = 64, M = 288, max_err = 8, w_bits = 24). The public inputs are pk_hash, config_hash,

C, w_hash and the nonce; the private inputs are the seed s, the M quantised weights and the configuration tags. Figure 3.5 shows the blocks. The circuit enforces that:

1. config_hash matches the circuit's own compiled parameters, so a strict registration cannot be proven with a lenient circuit;
2. C = H(COMMIT, s, pk_hash, config_hash), i.e. the prover knows the committed seed;
3. every quantised weight is in range and w_hash is the sponge of the private weights, binding the proof to the suspect model;
4. X and b are derived from the seed by the Poseidon streams, with strict bit decomposition so the prover cannot substitute an aliased field element;
5. each projection $\Sigma_j X_{ij} w_{q,j}$ is range-checked and its sign bit extracted;
6. the number of mismatches with b is at most max_err: the threshold is enforced inside the circuit; and
7. the nonce is constrained, so the proof is bound to one claim.

The circuit has **116,822 constraints** at T = 64, which fits a 2^{17} Powers of Tau. A security configuration with T = 256 and max_err = 32 is also compiled.

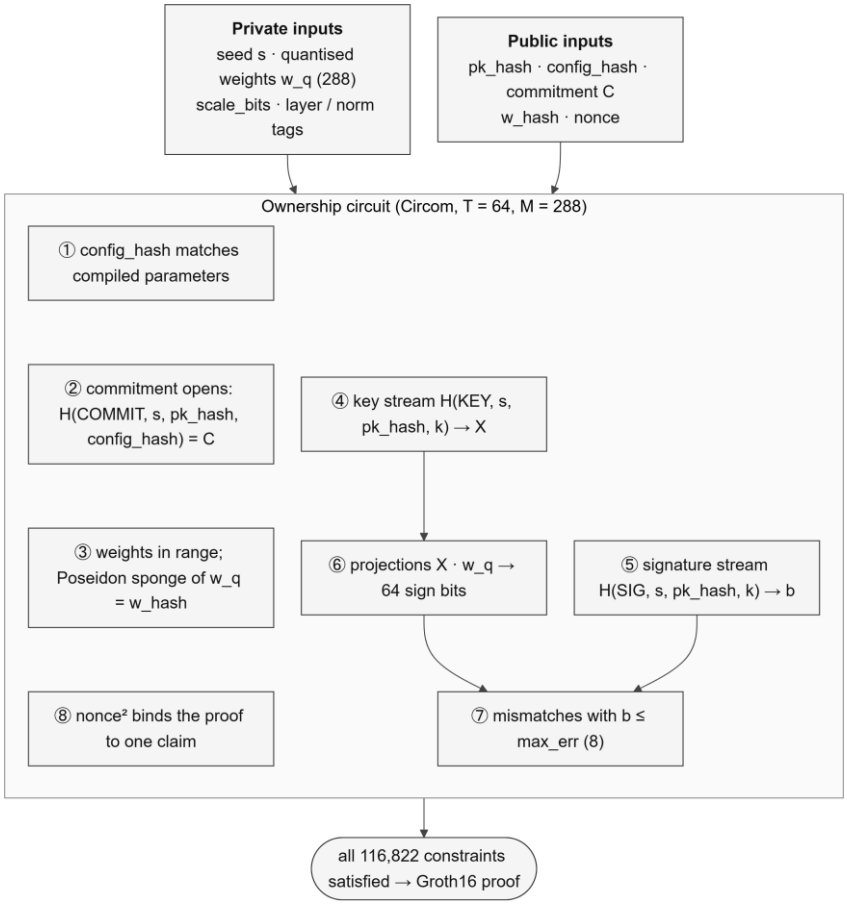


Figure 3.5: Block diagram of the ownership circuit

3.5.1 Trusted setup

Groth16 needs a structured reference string [23]. Phase 1 uses the public Perpetual Powers of Tau ceremony (pot28_0080, 80 contributions) [35], verified with SnarkJS on first download and pinned by hash. Phase 2 is a single contribution per circuit made with in-memory randomness that is never stored; its hash is recorded in a setup transcript in the repository, and the setup script refuses any shipped proving key whose SHA-256 does not match. The system is sound if at least one phase-1 contributor and the phase-2 contributor discarded their randomness.

3.6 Verification Procedure

A proof is accepted only if every check in Table 3.3 passes; each failure is reported with its own reason.

Table 3.3: Checks performed by the verifier

No.	Check	Defends against
1	Registration well-formed, derived fields recompute, owner signature valid	Spoofed or altered registration
2	Receipt signed by a trusted registry	Fake or back-dated timestamps
3	Registration earlier than the claim	Registering after a dispute starts
4	Claim refers to this registration	Proof for another registration
5	Claim refers to the SHA-256 of the suspect file held by the verifier	Proof about a different file
6–10	Public signals equal pk_hash, config_hash, C, w_hash (recomputed from the file) and nonce	Proof about another owner, configuration, model or claim (replay)
11	Correct number of public signals	Malformed proofs
12	Groth16 proof verifies under the registered configuration's key	Tampered or forged proofs

3.7 Software Implementation

Table 3.4 lists the technology stack.

Table 3.4: Technology stack

Layer	Technology	Purpose
Machine learning	Python 3.12, PyTorch 2.14.1 (CUDA 13.0), NumPy, ONNX, Keras adapter	Training, embedding, attacks, model loading
Cryptography	Poseidon (Python, checked against circomlibjs), Ed25519, scrypt + AES-256-GCM	Commitments, identities, encrypted secrets
Zero knowledge	Circom 2.2.3, circomlib 2.0.5, SnarkJS 0.7.6 (Groth16, BN254); EZKL 23.0.5 for research demos	Circuit, setup, proving, verification
SDK and CLI	sigil 0.2.0 (Python package)	Protocol, records, verify, adapters
Server	Python standard library HTTP server, SSE	Local API and job runner
Frontend	React, TypeScript, Vite; hand-built SVG charts	Dashboard (offline, no external assets)
On-chain (experiment)	Solidity verifier exported by SnarkJS, local chain	Verification cost in gas
Testing	pytest (226 tests in the SDK and server suites)	Regression and security tests

3.7.1 Python SDK and command-line tool

The *sigil* package contains modules for the protocol derivations (protocol), canonical signed records (records), ownership verification (verify), Ed25519 identities (identity),

the Poseidon hash (poseidon), encrypted secret storage (secretbox), embedding (embed), the Circom/SnarkJS bridge (zk) and framework adapters for PyTorch, Keras and ONNX (adapters). Records are canonical JSON with sorted keys and integer or string values only, so their hashes are reproducible. Model files are treated as untrusted: PyTorch files are loaded with weights_only=True and Keras files in safe mode.

3.7.2 Web dashboard

The dashboard has a landing page and an application with pages for an overview, embedding, verification, a robustness lab and the research demonstrations A–F. Figure 3.6 and Figure 3.7 give the data flow and use cases. The server accepts only whitelisted steps, checks the Host and Origin headers to block DNS rebinding and cross-site requests, requires JSON for POST requests, and runs one GPU job at a time.

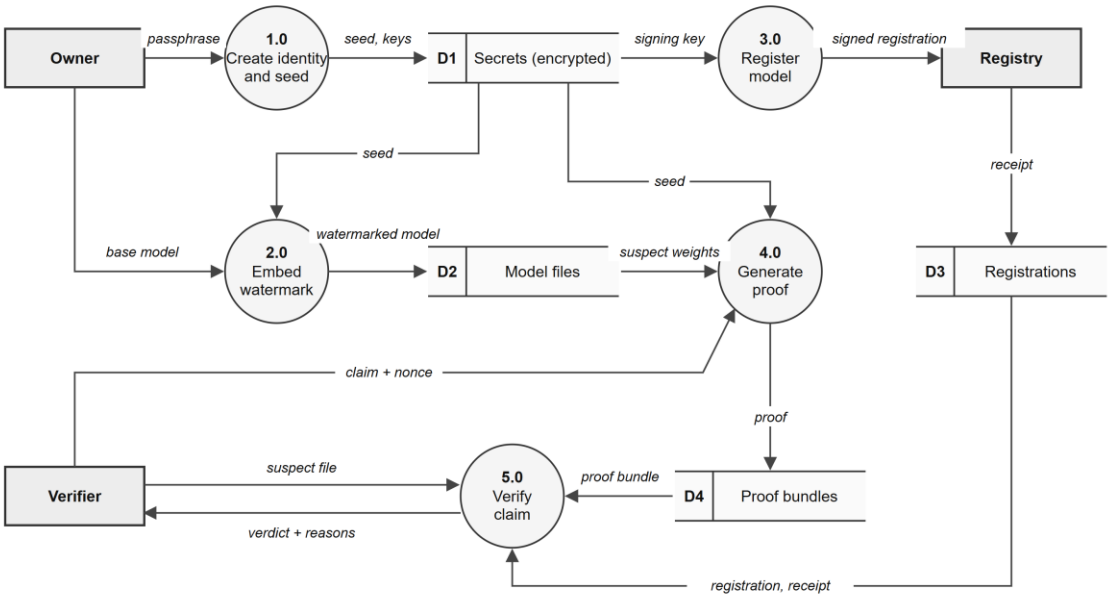


Figure 3.6: Data flow diagram (Level 1)

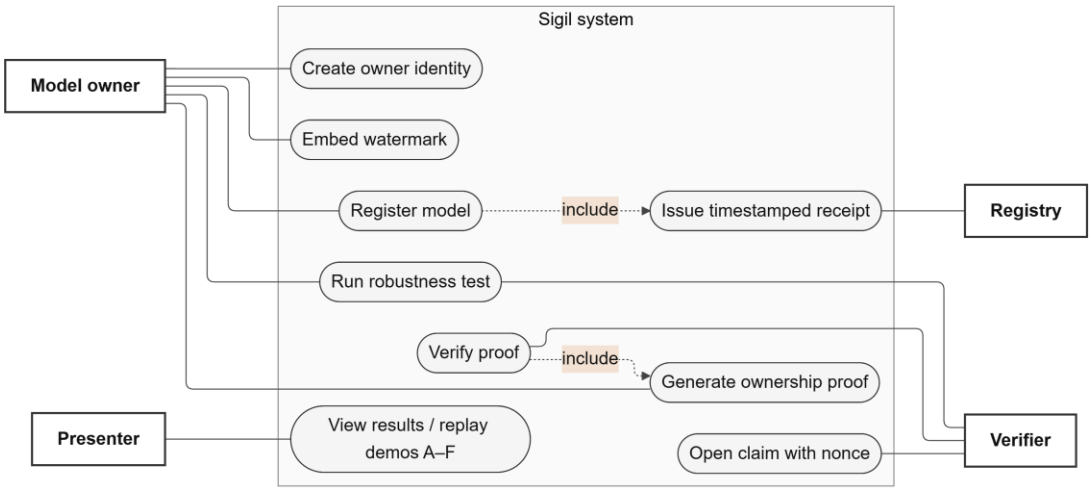


Figure 3.7: Use case diagram

3.8 Software Security Design

The software itself was threat-modelled with STRIDE; Table 3.5 summarises the main controls.

Table 3.5: STRIDE threat model of the software (summary)

Threat	Example	Control
Spoofing	Thief registers a model under another name	Ed25519-signed registrations; commitment bound to pk_hash
Tampering	Edit public signals to point at another model	Verifier recomputes every public signal; Groth16 check
Tampering	Swapped proving key in a shipped zip	SHA-256 of key checked against committed setup transcript
Repudiation	Owner denies registering	Signed registration and registry receipt
Information disclosure	Seed or key leaked in logs	Seeds encrypted at rest; key and signature never written; secret-scan test
Denial of service	Huge or zip-bomb model upload	Size limits, zip-bomb checks, per-step timeouts, single job
Elevation of privilege	Malicious pickle in a .pt file	torch.load(weights_only=True); Keras safe_mode
Elevation (web)	Another website POSTs to the local API	Host allow-list, Origin check, JSON-only POST, localhost-only CORS

Chapter 4

Experimental Setup and Results

This chapter reports the measured results. All numbers are read from the machine-readable result files produced by the demonstrations (demo/results/*.json and experiments/results/*.json). The experiments answer three research questions: **RQ1** (forgery: can a claim be forged with and without a commitment?), **RQ2** (robustness: does the watermark, and the ZK proof, survive removal attacks?) and **RQ3** (cost: is proving practical on one workstation?).

4.1 Experimental Environment

Table 4.1: Experimental environment

Component	Specification
CPU	AMD Ryzen 9 9950X3D, 16 cores / 32 threads
GPU	NVIDIA GeForce RTX 5080, 16 GB (compute capability 12.0)
RAM	32 GB DDR5
OS	Windows 11 Pro
Software	Python 3.12.15, PyTorch 2.14.1+cu130, Node.js 22, Circom 2.2.3, SnarkJS 0.7.6, EZKL 23.0.5
Dataset	MNIST handwritten digits (60,000 training / 10,000 test images) [36]
Model	Small CNN; watermark in layer conv2 (M = 288); T = 64 bits; $\theta = 8/64$

The demonstrations are organised as six steps, A to F, which can be run from the dashboard or the command line: A trains and watermarks the models, B runs removal attacks, C performs the forgery, D compares commitment variants, E runs the EZKL proofs, and F runs the full protocol.

4.2 Fidelity of the Watermark (Demo A)

Table 4.2 shows that embedding the watermark costs 0.03 percentage points of test accuracy, while the owner reads the signature with zero errors. On the clean (never watermarked) model the owner's key gives a bit error rate of 0.531, close to the 0.5 expected by chance.

Table 4.2: Accuracy and owner bit error rate

Model	Test accuracy (%)	Owner BER	Watermark detected
Clean (no watermark)	98.69	0.531	No
Watermarked (protocol v1)	98.66	0.000	Yes
Watermarked (protocol v2, Demo F)	98.57	0.000	Yes

4.3 Robustness to Removal Attacks (Demo B, RQ2)

The stolen model was attacked with a subset of the SoK attacks [7]. Table 4.3 and Figure 4.1 give the results. The watermark survives every pruning level, even 90–95% pruning that destroys the model (accuracy below 10%), int8 and int4 quantisation, and fine-tuning of all layers (FTAL) or retraining all layers (RTAL) for five epochs. Overwriting with the thief's own watermark leaves the owner's mark readable (BER 0.047) but creates a second valid mark, which is resolved by the registry timestamp (Section 4.6). **Feature permutation** of the layer's channels removes the watermark at no cost in accuracy (BER 0.438); the owner can undo it by realigning the layer against the original model before proving.

Table 4.3: Robustness of the watermark under removal attacks ($\theta = 0.125$)

Attack	Accuracy (%)	Owner BER	Survives
None (stolen as-is)	98.66	0.000	Yes
Prune 20% / 40%	98.60 / 97.28	0.000 / 0.000	Yes
Prune 65% / 80%	65.73 / 33.35	0.000 / 0.000	Yes
Prune 90% / 95%	9.41 / 8.52	0.016 / 0.016	Yes
int8 / int4 quantisation	98.65 / 98.66	0.000 / 0.000	Yes
FTAL 1 epoch / 5 epochs	98.58 / 98.81	0.000 / 0.000	Yes
RTAL 5 epochs	98.40	0.000	Yes
Overwrite (3 epochs)	98.62	0.047	Yes (ambiguous)
Feature permutation	98.66	0.438	No

Attack	Accuracy (%)	Owner BER	Survives
Permutation + owner realignment	98.66	0.000	Yes

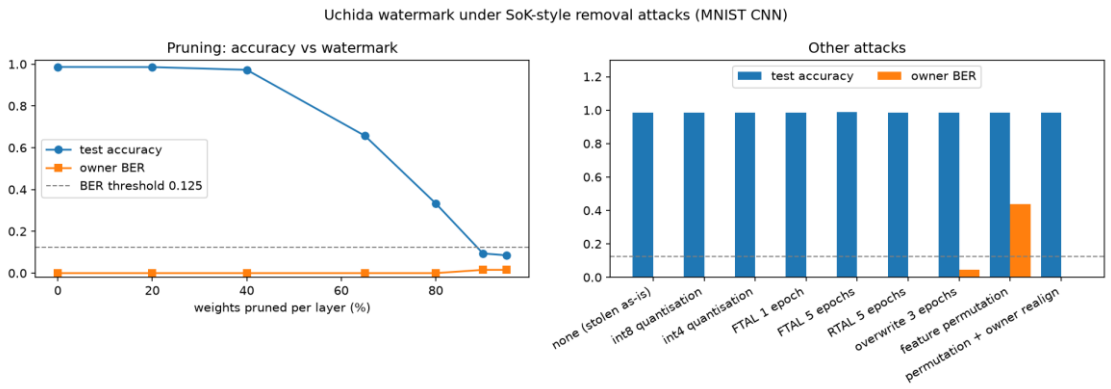


Figure 4.1: Accuracy and owner BER under removal attacks

4.4 Forgery Without a Commitment (Demo C, RQ1)

A thief who knows only the stolen weights fitted a counterfeit key that extracts the 64-bit encoding of the text "MALLORY!". The forged key achieved **BER 0.000** on the stolen model, and the same construction works on the clean model and on models that were never watermarked. A Kolmogorov–Smirnov test could not distinguish the forged key's entries from a genuine key's, and the forgery can be given margins as large as the owner's. Evaluated in PyTorch (not ZKROWNN's own code), the same sign-fitting also satisfies the extraction relation used by ZKROWNN [4], which is outside that paper's semi-honest threat model.

4.5 Committed-Seed Defence (Demo D, RQ1)

Table 4.4 compares three variants. Committing only to the key (D1) does not help: with a free signature the thief wins every one of 1,000 trials. Deriving the signature from the seed as well (D2) defeats the attack: in a search over 100,000 random seeds, none was accepted and the best seed still had 25% of its bits wrong.

Table 4.4: Forgery success under the three commitment variants

Variant	Description	Result
D0	Owner verifies against the published commitment	Commitment matches, BER 0.000, verified
D1	Key derived from committed seed, signature free	Thief wins 1,000 / 1,000

Variant	Description	Result
D2	Key and signature both derived from committed seed	0 / 100,000 seeds accepted; best BER 0.250; mean BER 0.500

The probability that one random seed is accepted is $P[\mathrm{Bin}(64, ½) \leq 8] \approx 2.8 \times 10^{-10}$, about $2^{-31.7}$. At the measured 9,155 seeds per second on one core, a thief needs about 2.5×10^9 tries, roughly 76 single-core hours, for a 50% chance. This is feasible on a GPU, which is why the registry timestamp remains necessary at T = 64; at T = 256 the work rises to about 2^{120} tries. Figure 4.2 shows the distribution of the thief's bit error rates.

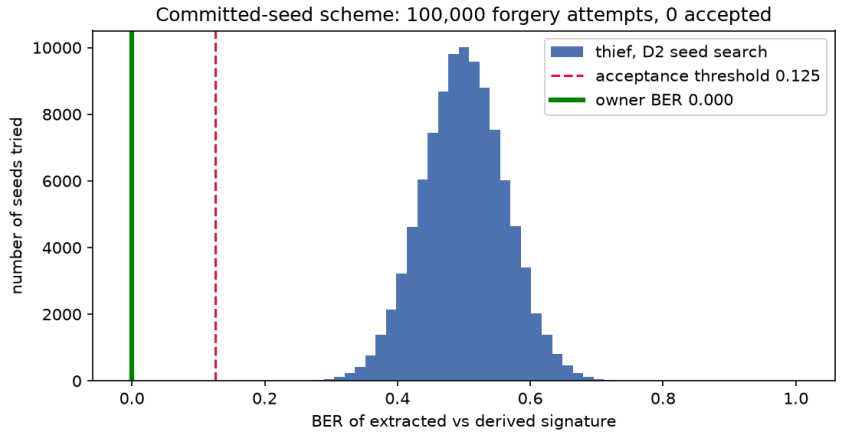


Figure 4.2: Distribution of BER over the thief's seed search (D2)

4.6 ZK Proofs with EZKL (Demo E)

The early experiments used EZKL [27] to prove the extraction. Table 4.5 shows the cost. With private, uncommitted parameters (E1), the owner and the thief both obtain valid proofs, and in E2 the **thief's forged proof verified under the owner's verification key**: the two keys were identical. Hashing the parameters into the public output (E3) separates owner and thief, but raises the circuit size from 2^{15} to 2^{20} rows and the proving time from 0.7 s to 39–47 s, with a 9.8 GiB proving key. E3 also only commits to the key and signature directly (the D1 level), which motivated the dedicated Circom circuit.

Table 4.5: Cost of EZKL proofs (owner and thief runs)

Metric	E1: private parameters	E3: Poseidon-hashed parameters
Circuit size (logrows)	15	20
Setup time	0.70–0.72 s	33.4–34.2 s
Proving time	0.68 s	39.1–46.9 s
Verification time	22 ms	0.56–0.58 s

Metric	E1: private parameters	E3: Poseidon-hashed parameters
Proof size	≈ 60 KiB	≈ 64.5 KiB
Proving key size	132 MiB	9.8 GiB
Peak RAM	0.6 GiB	2.6 GiB
Owner and thief distinguishable?	No (forgery verifies under owner VK)	Yes (different public hash)

4.7 End-to-End Protocol (Demo F)

Demo F runs the complete protocol through the SDK. The owner registered the model (registry receipt timestamp 2026-10-04 21:43 UTC), and the verifier accepted the owner's proof with all twelve checks passing. Witness generation took 0.42 s and proving 2.23 s. The thief could not produce a proof: with their own seed and key, 35 of 64 bits were wrong, and they cannot open the owner's commitment without the seed. Table 4.6 shows how misuse attempts are rejected, each with its own reason.

Table 4.6: Rejection of misuse attempts in protocol v2

Attempt	Accepted	Reason reported by the verifier
Owner's genuine proof	Yes	All 12 checks pass
Thief with own seed and key	No proof	35/64 bits wrong (> 8)
Thief using owner's commitment	No proof	Cannot open the commitment
Proof replayed for another claim	No	Nonce mismatch (replay)
Proof checked against a different model	No	w_hash mismatch
Tampered proof bytes	No	zk-SNARK does not verify
Receipt from an untrusted registry	No	Receipt not from a trusted registry; registration not earlier than claim

Table 4.7 repeats the robustness test inside the ZK protocol: the proof is still accepted after heavy pruning and fine-tuning, and permutation again defeats the white-box mark.

Table 4.7: Robustness of the ZK ownership proof (protocol v2)

Attack	Accuracy (%)	Bit errors (of 64)	Proof accepted
Prune 65%	50.26	0	Yes
Prune 90%	2.01	1	Yes
Fine-tune all layers, 5 epochs	98.83	0	Yes
Feature permutation	98.57	39	No (> 8 errors)

Overwriting dispute. The thief embedded and registered their own watermark in the stolen model the next day (2026-10-05 06:28 UTC). On the overwritten copy both proofs verify: the owner's with 1 bit error and the thief's with 0. The registry timestamps decide the dispute in the owner's favour, as specified in Section 3.4.3.

4.8 Cost of the Circom Circuit (RQ3)

Table 4.8: Cost of the protocol v2 ownership circuit

Metric	Value
Constraints (T = 64, M = 288)	116,822
Powers of Tau size	2^{17}
Witness generation	0.42 s
Groth16 proving	2.23 s
Public signals	5 (pk_hash, config_hash, C, w_hash, nonce)
On-chain verifier: deploy gas	430,402
On-chain verifier: verification gas	243,225
On-chain: tampered public signal	Rejected

Compared with the EZKL E3 proof, which commits only to the key and signature, the Circom circuit proves the stronger statement (seed opening, in-circuit derivation, threshold, model binding and nonce) about 18–21 times faster. The on-chain experiment deployed the SnarkJS-exported Solidity verifier on a local development chain and confirmed that a proof verifies and that a tampered public signal is rejected.

4.9 Web Dashboard

The dashboard presents every result in this chapter and can either run each step live on the workstation or show the recorded results of earlier runs. The screenshots below were captured from the running application (http://127.0.0.1:8765) with the saved results loaded. Every page shows a model bar at the top, where the model under test is chosen, and a side navigation listing the main pages and research demos A–F.

Figure 4.3 shows the landing page, which introduces the product and its supported model formats (.pt, .h5, .onnx). The overview in Figure 4.4 summarises the headline numbers: proving time, accuracy cost, forgery attempts accepted and attacks survived, together with the model list and the proof pipeline.

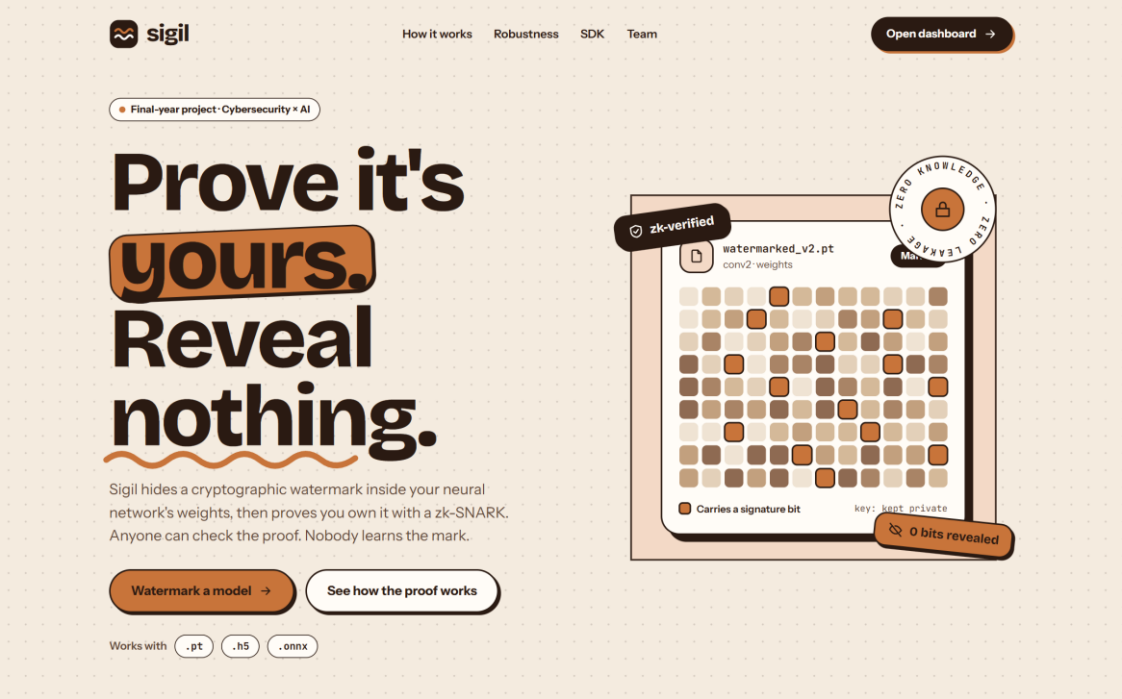


Figure 4.3: Dashboard: landing page



Figure 4.4: Dashboard: overview with headline results

The Embed page (Figure 4.5) walks through the owner's steps — owner key, committed seed, embedding and registration — and reads the watermark back from the selected model. The Verify page (Figure 4.6) runs the verifier's checks on a model and shows the verdict, here "Ownership proven", with each check listed.

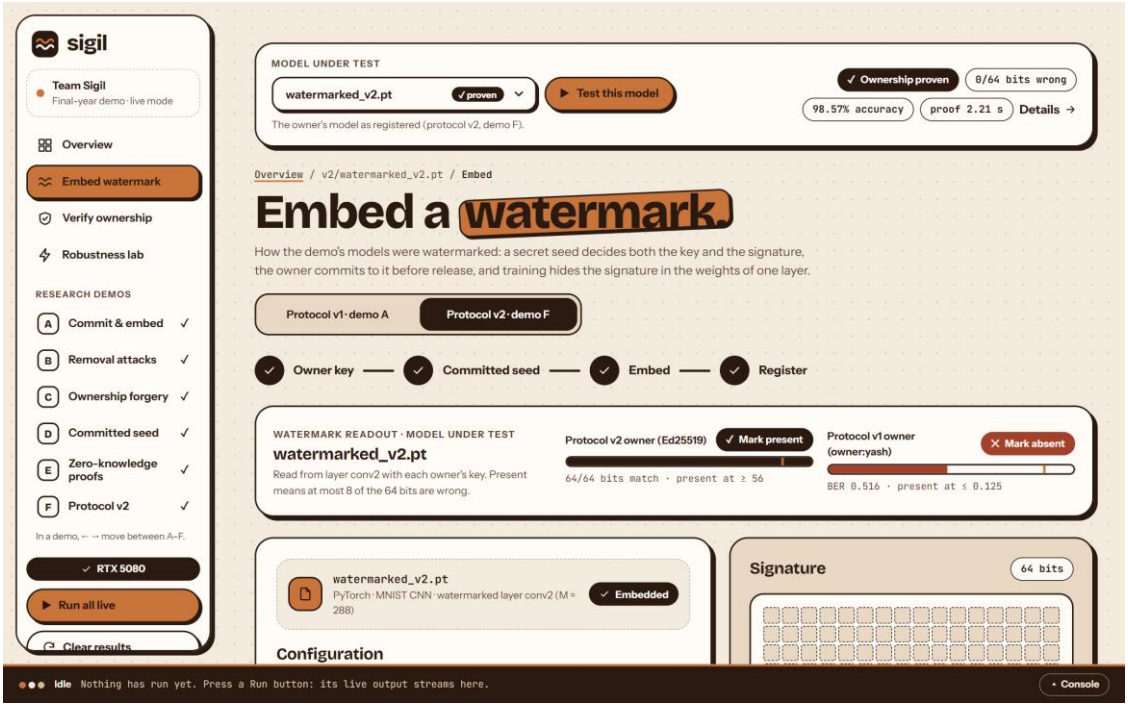


Figure 4.5: Dashboard: embedding the watermark

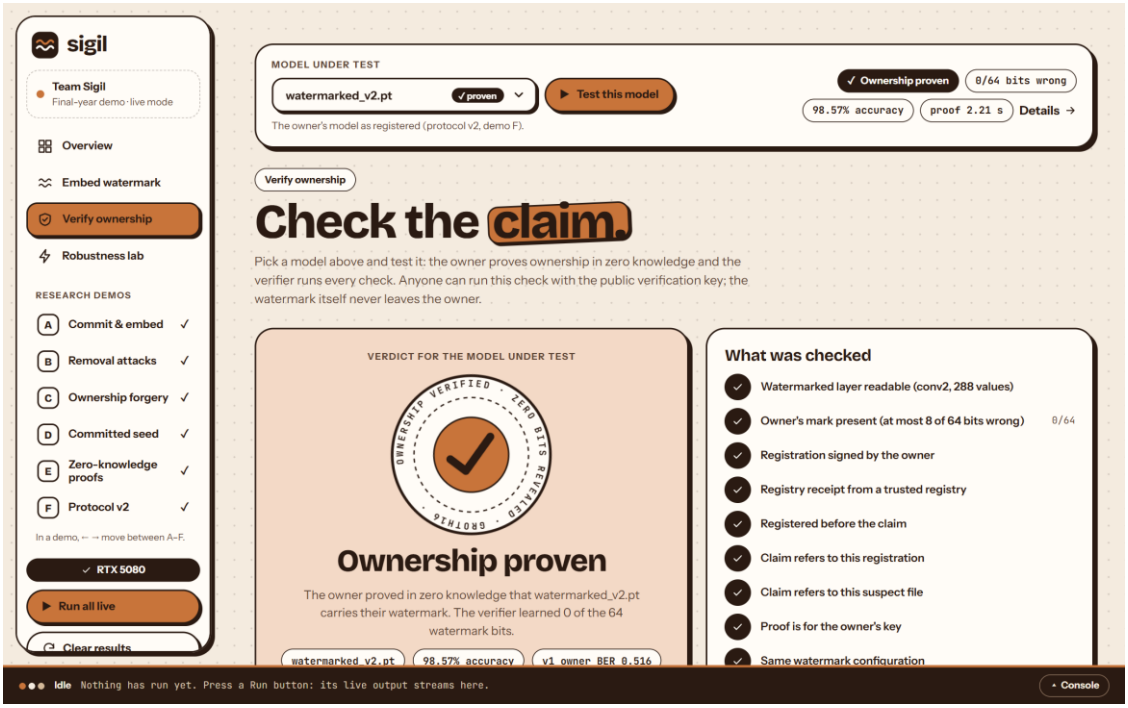


Figure 4.6: Dashboard: ownership verification verdict

The Robustness lab (Figure 4.7) lets the user attack the owner's model as a thief would and test whether the watermark survives. The research-demo pages present each experiment with its result: Figure 4.8 shows the forgery of Demo C, Figure 4.9 the committed-seed comparison of Demo D, and Figure 4.10 the zero-knowledge proof of protocol v2 (Demo F).

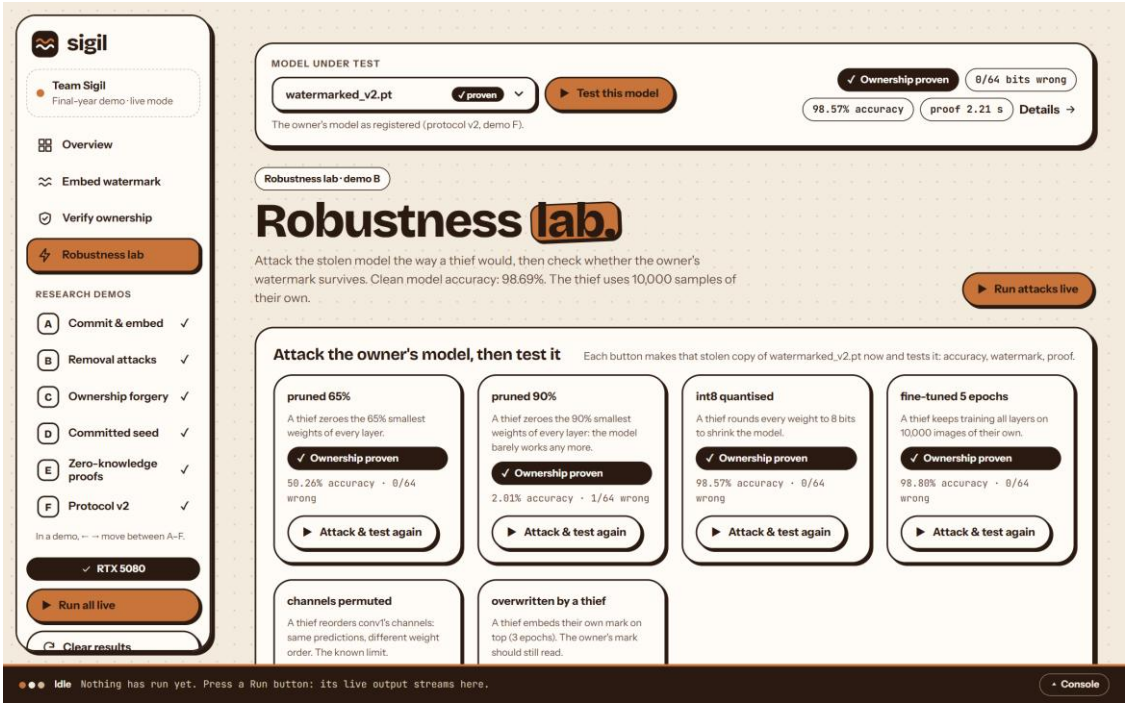


Figure 4.7: Dashboard: robustness lab

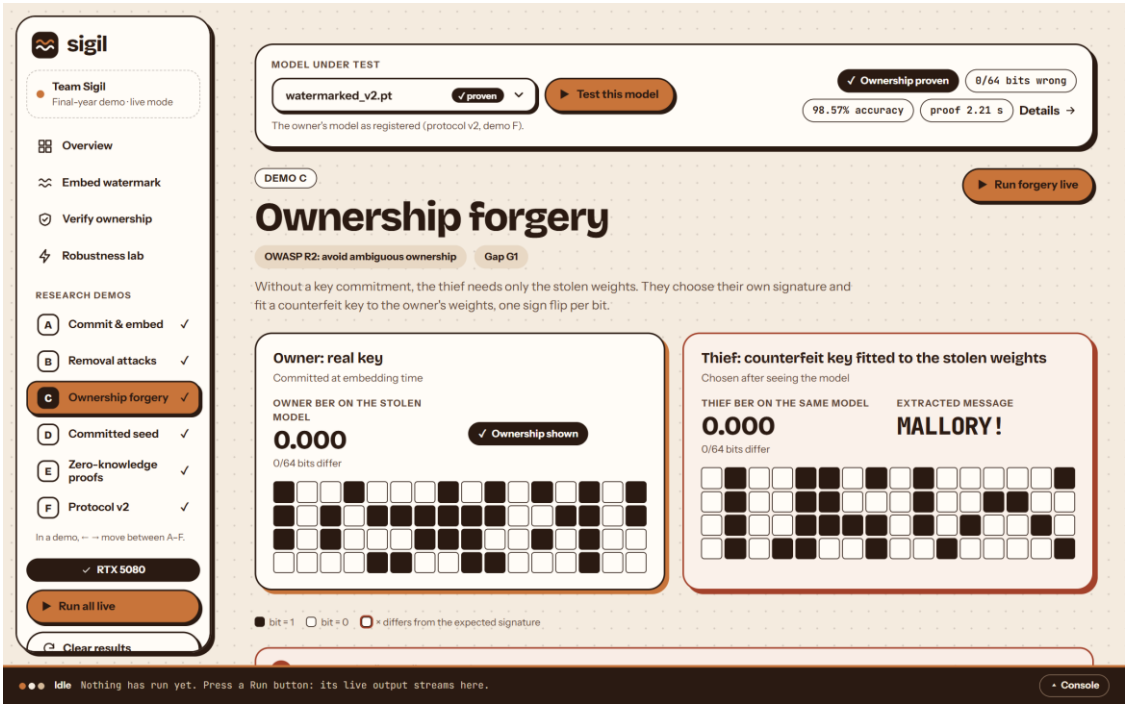


Figure 4.8: Dashboard: Demo C, ownership forgery without a commitment

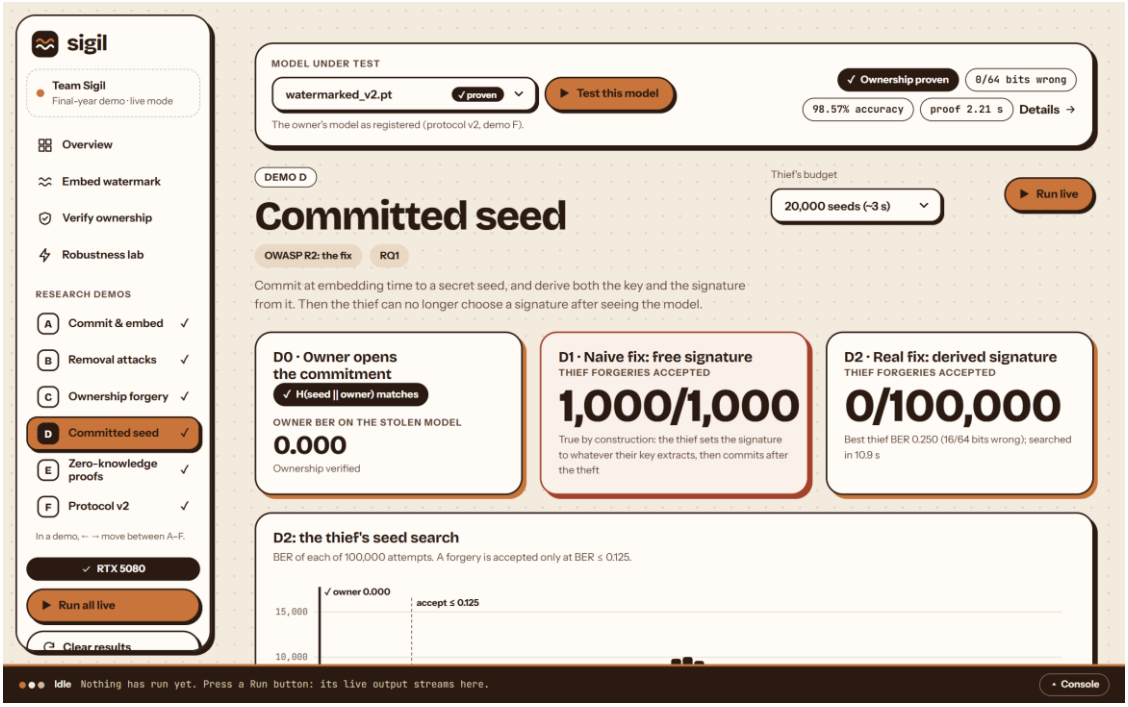


Figure 4.9: Dashboard: Demo D, committed-seed defence

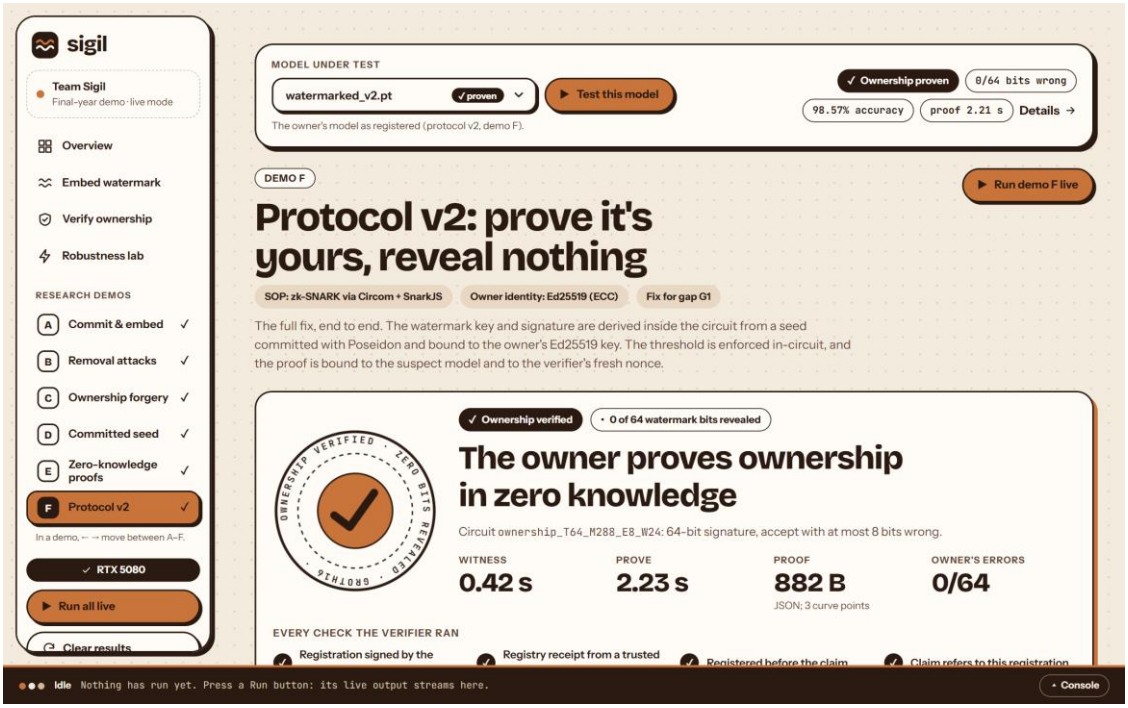


Figure 4.10: Dashboard: Demo F, protocol v2 ownership proof

4.10 Discussion and Limitations

- **RQ1 (forgery).** Without a commitment a claim can be forged with zero bit errors, and the forged ZK proof verifies under the owner's key. Deriving both key and signature from the committed seed reduces forgery to a seed search with about $2^{-31.7}$ success per try at T = 64.

- **RQ2 (robustness).** The watermark and the ZK proof survive pruning, quantisation and fine-tuning. Permutation removes the white-box mark; owner-side realignment restores it, but needs the original model.
- **RQ3 (cost).** A proof takes about 2.2 s and fits easily in 32 GB of RAM on a single workstation.
- **Threshold.** $\theta = 8/64$ is indicative. A handful of null models cannot give a calibrated false-positive rate; more independent models are needed.
- **Security level.** $T = 64$ gives about 2^{32} work, which a GPU can do; $T = 256$ is the security configuration.
- **Trusted setup.** Phase 2 has a single contributor; a multi-party ceremony is out of scope.
- **Scope.** Results are on MNIST with a small CNN; white-box access to the suspect model is required.

Chapter 5

Conclusion

5.1 Conclusion

This project set out to build an open, zero-knowledge ownership-verification system for neural-network watermarks, aligned with the OWASP AI Model Watermarking project. Our survey and experiments showed that hiding the watermark key in a ZK proof is not enough on its own: without a prior commitment, anyone holding the model can fit a counterfeit key, and the forged proof verifies under the genuine owner's verification key. Committing to the key alone does not fix this, because the signature can still be chosen to match.

We designed and implemented Sigil, in which the key and the signature are both derived from a seed committed at embedding time, bound to an Ed25519 identity and a registry timestamp. A Circom circuit of 116,822 constraints proves, with Groth16, that the prover knows the committed seed, that the derived key extracts the derived signature from the suspect model's weights within the threshold, and that the proof belongs to one model and one claim. On MNIST the watermark costs 0.03 percentage points of accuracy, survives pruning, quantisation and fine-tuning, and resists a 100,000-seed forgery search; replayed, misdirected and tampered proofs are rejected with specific reasons, and overwriting disputes are settled by registry timestamps. A proof takes about 2.2 s on one workstation. The system is delivered as a Python SDK, a command-line tool and a local web dashboard. Feature permutation remains a known weakness of the white-box watermark.

5.2 Future Scope

- **Backend and database.** A FastAPI backend with a PostgreSQL database for multi-user registration, verification history and a hash-chained audit log, as planned in the Statement of Purpose.
- **Stronger security configuration.** Adopt T = 256 bits by default, and calibrate the threshold with a larger set of independently trained null models.
- **Permutation robustness.** Make the extraction invariant to channel permutation, or prove the owner's realignment inside the circuit.
- **Black-box coverage.** Add a trigger-set watermark derived from the committed seed to cover query-only model extraction.
- **Trusted setup.** Run a multi-party phase-2 ceremony, or move to a proof system with a universal or transparent setup.
- **Larger models.** Extend the evaluation to CIFAR-10 and ResNet models and investigate fingerprinting of small language models.
- **OWASP contribution.** Propose the protocol and reference implementation to the OWASP AI Model Watermarking project.

References

[1] OWASP Foundation, "ML05:2023 Model Theft," OWASP Machine Learning Security Top 10. [Online]. Available: https://owasp.github.io/www-project-machine-learning-security-top-10/docs/ML05_2023-Model_Theft

[2] OWASP AI Exchange, "#MODEL WATERMARKING," in OWASP AI Exchange, PDF export generated 3 Oct. 2026, pp. 230–231. [Online]. Available: https://owaspai.org/go/modelwatermarking

[3] Y. Uchida, Y. Nagai, S. Sakazawa, and S. Satoh, "Embedding watermarks into deep neural networks," in Proc. ACM Int. Conf. Multimedia Retrieval (ICMR), 2017, pp. 269–277, doi: 10.1145/3078971.3078974.

[4] N. Sheybani, Z. Ghodsi, R. Kapila, and F. Koushanfar, "ZKROWNN: Zero knowledge right of ownership for neural networks," in Proc. 60th ACM/IEEE Design Automation Conf. (DAC), 2023, pp. 1–6, doi: 10.1109/DAC56929.2023.10247798.

[5] OWASP Foundation, "OWASP AI Model Watermarking (Incubator project)," [Online]. Available: https://owasp.org/projects/ai-model-watermarking. Accessed: Oct. 2026.

[6] OWASP Foundation, "www-project-ai-model-watermarking," GitHub repository. [Online]. Available: https://github.com/owasp/www-project-ai-model-watermarking. Accessed: Oct. 2026.

[7] N. Lukas, E. Jiang, X. Li, and F. Kerschbaum, "SoK: How robust is image classification deep neural network watermarking?," in Proc. IEEE Symp. Security and Privacy (SP), 2022, pp. 787–804, doi: 10.1109/SP46214.2022.9833693.

[8] OWASP Foundation, "SecureML," GitHub repository. [Online]. Available: https://github.com/OWASP/SecureML. Accessed: Oct. 2026.

[9] OWASP GenAI Security Project, "LLM10:2025 Unbounded Consumption," OWASP Top 10 for LLM Applications 2025. [Online]. Available: https://genai.owasp.org/llmrisk/llm102025-unbounded-consumption/

[10] H. Jia, C. A. Choquette-Choo, V. Chandrasekaran, and N. Papernot, "Entangled watermarks as a defense against model extraction," in Proc. 30th USENIX Security Symp., 2021, pp. 1937–1954.

[11] B. D. Rouhani, H. Chen, and F. Koushanfar, "DeepSigns: A generic watermarking framework for IP protection of deep learning models," arXiv:1804.00750, 2018.

[12] Y. Yao, J. Song, and J. Jin, "Hashed watermark as a filter: Defeating forging and overwriting attacks in weight-based neural network watermarking," in Proc. AAAI Conf. Artificial Intelligence, 2026; arXiv:2507.11137.

[13] Y. Adi, C. Baum, M. Cisse, B. Pinkas, and J. Keshet, "Turning your weakness into a strength: Watermarking deep neural networks by backdooring," in Proc. 27th USENIX Security Symp., 2018, pp. 1615–1631.

[14] H. Li, E. Wenger, S. Shan, B. Y. Zhao, and H. Zheng, "Piracy resistant watermarks for deep neural networks," arXiv:1910.01226v3, 2020.

[15] B. Choi, S. Wang, I. Choi, and K. Sun, "ChainMarks: Securing DNN watermark with cryptographic chain," in Proc. 20th ACM Asia Conf. Computer and Communications Security (ASIA CCS), 2025, pp. 442–455, doi: 10.1145/3708821.3736214.

[16] T. Gloaguen, N. Jovanović, R. Staab, and M. Vechev, "Towards watermarking of open-source LLMs," arXiv:2502.10525, 2025.

[17] L. Fan, K. W. Ng, and C. S. Chan, "Rethinking deep neural network ownership verification: Embedding passports to defeat ambiguity attacks," in Advances in Neural Information Processing Systems (NeurIPS), 2019; arXiv:1909.07830.

[18] J. Liu, R. Zhang, S. Szyller, K. Ren, and N. Asokan, "False claims against model ownership resolution," in Proc. 33rd USENIX Security Symp., 2024, pp. 6885–6902.

[19] P. Bui and T. Tran, "Beyond robustness: On the unforgeability trade-offs in statistical DNN watermarking," in Proc. 2nd Int. Conf. Cryptography and Information Security (VCRIS), 2025, pp. 1–6, doi: 10.1109/VCRIS68011.2025.11250570.

[20] M. Russinovich, Y. Cai, and A. Salem, "Hey, that's my model! Introducing Chain & Hash, an LLM fingerprinting technique," in Proc. Int. Conf. Learning Representations (ICLR), 2026; arXiv:2407.10887.

[21] S. Goldwasser, S. Micali, and C. Rackoff, "The knowledge complexity of interactive proof-systems," in Proc. 17th Annu. ACM Symp. Theory of Computing (STOC), 1985, pp. 291–304, doi: 10.1145/22145.22178.

[22] B. Parno, J. Howell, C. Gentry, and M. Raykova, "Pinocchio: Nearly practical verifiable computation," in Proc. IEEE Symp. Security and Privacy, 2013, pp. 238–252, doi: 10.1109/SP.2013.47.

[23] J. Groth, "On the size of pairing-based non-interactive arguments," in Advances in Cryptology – EUROCRYPT 2016, LNCS, Springer, 2016, pp. 305–326, doi: 10.1007/978-3-662-49896-5_11.

[24] L. Grassi, D. Khovratovich, C. Rechberger, A. Roy, and M. Schofnegger, "Poseidon: A new hash function for zero-knowledge proof systems," in Proc. 30th USENIX Security Symp., 2021, pp. 519–535.

[25] M. Bellés-Muñoz, M. Isabel, J. L. Muñoz-Tapia, A. Rubio, and J. Baylina, "Circom: A circuit description language for building zero-knowledge applications," IEEE Trans. Dependable and Secure Computing, vol. 20, no. 6, pp. 4733–4751, 2023, doi: 10.1109/TDSC.2022.3232813.

[26] iden3, "snarkjs: zkSNARK implementation in JavaScript & WASM," GitHub repository. [Online]. Available: https://github.com/iden3/snarkjs. Version 0.7.6 used.

[27] zkonduit, "EZKL," GitHub repository and PyPI package (v23.0.5). [Online]. Available: https://github.com/zkonduit/ezkl

[28] R. Zhang, N. Javidnia, N. Sheybani, and F. Koushanfar, "Robust and secure code watermarking for large language models via ML/crypto codesign," arXiv:2502.02068v3, 2026 (ACM Trans. AI Security and Privacy).

[29] A. Ramakrishnan, S. Agarwal, S. Selvanayagam, and K. Singh, "ZK-WAGON: Imperceptible watermark for image generation models using ZK-SNARKs," in Proc. 5th Int. Conf. AI-ML Systems (AIMLSystems), 2025, pp. 194–198, doi: 10.1109/AIMLSystems67835.2025.11387031.

[30] W. Yang, Y. Yin, G. Zhu, H. Gu, L. Fan, X. Cao, and Q. Yang, "FedZKP: Federated model ownership verification with zero-knowledge proof," arXiv:2305.04507v2, 2023.

[31] H. Sun, J. Li, and H. Zhang, "zkLLM: Zero knowledge proofs for large language models," in Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS), 2024, pp. 4405–4419, doi: 10.1145/3658644.3670334.

[32] J. Xu, F. Wang, M. Ma, P. W. Koh, C. Xiao, and M. Chen, "Instructional fingerprinting of large language models," in Proc. NAACL-HLT, 2024, pp. 3277–3306, doi: 10.18653/v1/2024.naacl-long.180.

[33] B. Liu, S. Wang, Y. Zhu, X. Yu, J. Huang, L. Du, H. Pei, and W. Luo, "Implicit identity technologies for LLMs: Fingerprinting and watermarking across datasets, models, and generated content," in Proc. IJCAI-ECAI, 2026; arXiv:2605.29245.

[34] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang, "High-speed high-security signatures," J. Cryptographic Engineering, vol. 2, pp. 77–89, 2012, doi: 10.1007/s13389-012-0027-1.

[35] Privacy & Scaling Explorations, "Perpetual Powers of Tau," trusted-setup ceremony (file pot28_0080, 80 contributions). [Online]. Available: https://github.com/privacy-scaling-explorations/perpetualpowersoftau

[36] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," Proc. IEEE, vol. 86, no. 11, pp. 2278–2324, 1998, doi: 10.1109/5.726791.

Appendix-I: Protocol Configuration and Notation

Public configuration fixed at embedding time (demo values):

Field	Value	Meaning
T	64 (256 for security)	Signature length in bits
M	288	Length of the Uchida vector (in_channels × kh × kw)
layer	conv2	Watermarked layer
scale_bits	16	Fixed point: $w_q = \text{round}(w \cdot 2^{16})$
w_bits	24	Range: $\lvert w_q \rvert < 2^{24}$
max_err	8	Accept iff at most 8 of T bits disagree
PRG_BITS	200	Low bits taken from each Poseidon output
Field	BN254 scalar field	Arithmetic of the circuit
Domain tags	KEY 1, SIG 2, COMMIT 3, W 4, PK 5, CFG 6	Separate the uses of H

Records: *sigil.registration/v2* (signed by the owner), *sigil.receipt/v2* (signed by the registry) and *sigil.claim/v2* (issued by the verifier, containing the nonce). All are canonical JSON with integers and strings only.

Appendix-II: Copies of the Published Papers

[Insert copies of any papers published or submitted from this project here.]

Appendix-III: Experimental Data

Key parity study (WM-02). Mean over five seeds, three training epochs each, comparing Uchida's Gaussian key with the ±1 key used by the circuit. Values are accuracy (%) / owner BER.

Condition	Gaussian key	±1 key
No attack	98.68 / 0.000	98.71 / 0.000
Clean-model BER (owner key)	0.516	0.544
Prune 65%	76.17 / 0.000	87.02 / 0.000
Prune 90%	16.77 / 0.028	12.17 / 0.013
int8 / int4	98.68 / 0.000 ; 98.56 / 0.000	98.70 / 0.000 ; 98.57 / 0.000
FTAL 5 / RTAL 5	98.92 / 0.000 ; 98.48 / 0.000	98.75 / 0.000 ; 98.45 / 0.000
Overwrite 3 epochs	98.74 / 0.013	98.64 / 0.034
Permutation	98.68 / 0.456	98.71 / 0.525

Raw results are in the project repository: demo/results/ (train, attacks, forgery, committed_seed, vk_crosscheck, ezkl_*, protocol_v2) and experiments/results/ (wm02_parity, zk08_onchain).

Appendix-IV: Installation and User Guide

Prerequisites: Python 3.10+, Node.js 20.19+, internet during setup. An NVIDIA GPU is optional.

Setup (Windows): from the project folder run *setup.ps1 --demo* (add --*gpu* for CUDA). The script creates a Python 3.12 virtual environment, installs PyTorch and the SDK, installs Circom and SnarkJS, unpacks and checks the proving keys, and runs a self-test.

Verify the deployment: *.venv\Scripts\python scripts\verify_deploy.py* must end with ALL CHECKS PASSED. It makes and verifies a real Groth16 proof.

Run: *.venv\Scripts\python demo\server.py --open* and open http://127.0.0.1:8765. Without the ML stack, *python demo/server.py --simulate* replays recorded runs.

The illustrated user guide (Sigil-User-Guide.pdf) in the repository documents every screen.